

手掌跟踪实验手册

文档版本

01

发布日期

2024-07-05



版权所有 © 华为云计算技术有限公司 2023。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为云计算技术有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为云计算技术有限公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

手掌跟踪

第一章 实验准备

1.1. 实验描述

本实验的目标是基于 `cvzone` 计算机视觉库实现机器狗的目标追踪如手掌跟踪，识别手掌的关键点，例如手指尖和关节。通过这些关键点计算出实验中所需要的距离和角度来进行进一步的操作。在实验过程中，我们在对机器狗的精准控制之上，对手掌关键点进行解算，通过得到的数据来调整机器狗的动作。

1.2. 实验目的

1. 学生将学习如何捕获实时视频流，并使用计算机视觉库进行图像处理和分
析
2. 学习如何使用 `cvzone` 库和 `MediaPipe Hands` 模型进行手掌检测与关键点提
取
3. 理解手掌关键点的含义及其在计算机视觉中的应用
4. 学生将了解如何根据手掌关键点计算手掌的距离和角度

1.3. 实验建议

1. 前置课程：

Python 编程: 学生应具备基本的 `Python` 编程知识，包括数据结构、函数和面向对象编程。

计算机视觉基础: 建议学生了解一些基础的计算机视觉知识，如图像处理、特征提取等。

OpenCV 库: 学生应熟悉 `OpenCV` 库的基本用法，如如何读取和显示图

像。

数据挖掘：数据挖掘是学生从大量数据中自动或半自动地发现模式和知识的过程。

2.实验前准备：

安装必要的软件：确保所有学生的计算机上已安装 Python、OpenCV 和 cvzone 库。

硬件准备：每个学生或实验组应配备一台带摄像头的电脑。如果使用外接摄像头，3.请确保其正常工作。

环境设置：建议在统一的虚拟环境中安装所有依赖项，以避免版本不兼容的问题。

4.实验步骤详解：

代码讲解：在实验开始前，详细讲解代码的每一部分，确保学生理解每一步的操作和原理。

关键点标注：解释手掌关键点的具体含义及其在实际应用中的作用。

5.调试与优化：

逐步调试：鼓励学生在代码开发过程中逐步调试，逐行检查输出和结果，及时发现和解决问题。

性能优化：讨论如何优化手掌检测的性能，如调整检测器的参数、使用更高效的算法等。

6.结果验证：

测试用例：提供一些典型的测试用例，让学生验证他们的程序能否正确

识别手掌并控制机器狗。

错误处理：讨论常见错误及其解决方法，如手掌检测不准确、机器狗控制失灵等

1.4. 实验环境

本次实验使用的硬件包括：PC 机、昇腾 Atlas 200I DK 开发板、摄像头。实验开始前，请按照如下步骤将各硬件连接：

1. 使用数据线将开发板连接至 PC 机；
2. 将摄像头连接至开发板；
3. 给开发版供电，确保其能正常开机；

1.5. 背景知识

1.5.1. OpenCV (Open Source Computer Vision Library)



是一个开源的计算机视觉和机器学习软件库。它由 Intel 公司在 1999 年首次发布，现在由 OpenCV 基金会维护。OpenCV 旨在实现实时的计算机视觉，包括但不限于以下领域：

图像处理：包括图像的基本操作，如读取、显示、转换格式、调整大小、旋转、裁剪等。

特征检测与描述：用于识别图像中的角点、边缘、直线、轮廓等特征，并提取它们的描述符。

图像分割：将图像分割成多个区域或对象，例如通过阈值分割、区域生长、水印切割等方法。

图像分类与识别：使用机器学习算法对图像进行分类和识别。

目标检测：识别图像中的对象，并确定它们的位置和大小。

运动估计：分析图像序列中的运动，如光流法。

立体视觉：处理立体图像，进行深度估计。

人机交互：实现手势识别、面部识别等交互功能。

机器学习和深度学习：集成了多种机器学习算法和深度学习框架，如 SVM、随机森林、神经网络等。

实时视频处理：支持实时视频捕获和处理。

OpenCV 支持多种编程语言，包括 C++、Python、Java 等，这使得它在学术研究和商业应用中都非常受欢迎。它广泛应用于自动驾驶、机器人、面部识别、医疗图像分析、视频监控、增强现实等领域。

OpenCV 的安装相对简单，可以通过包管理器（如 pip for Python）或从源代码编译来安装。它还提供了大量的文档和教程，帮助用户快速上手和深入学习。

由于 OpenCV 是一个庞大的库，涵盖了计算机视觉的各个方面，这里只能提供一个概览。如果你对某个特定领域感兴趣，可以进一步探索 OpenCV 提供的详细文档和示例代码。

1.5.2. CVZone



一个由 Adrian Rosebrock 创建的开源库，专门用于计算机视觉任务，特别是那些使用 Python 和 OpenCV 的任务。这个库旨在简化一些常见的计算机视觉操作，使得开发者可以更容易地实现复杂的视觉处理功能

CVZone 的特点:

1. **简化操作：**CVzone 提供了一系列函数和类，帮助开发者快速实现如手掌跟踪、面部识别等高级功能。
2. **面向对象：**CVzone 采用面向对象的设计，使得代码更加模块化和易于管理。
3. **易于使用：**CVzone 的 API 设计直观易懂，即使是初学者也能够快速上手。
4. **集成 OpenCV：**CVzone 与 OpenCV 紧密集成，可以无缝地使用 OpenCV 的强大功能。
5. **实时性能：**CVzone 优化了性能，使得它能够支持实时的计算机视觉应用。
6. **示例丰富：**CVzone 提供了大量的示例代码，帮助开发者理解如何使用库中的各种功能。
7. **社区支持：**作为一个开源项目，CVzone 拥有一个活跃的社区，开发者可以在这里获取帮助和分享经验。

8. **文档齐全**：CVzone 提供了详细的文档，包括安装指南、API 参考和教程。
9. **持续更新**：CVzone 作为一个活跃的项目，会定期更新，增加新功能和修复已知问题。
10. **适用于教育和研究**：CVzone 不仅适用于商业项目，也非常适合教育和研究领域，帮助学生和研究人员快速实现和测试计算机视觉算法

1.5.3. MediaPipe

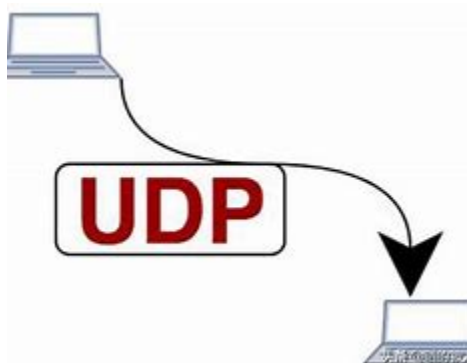


一个由 Google 开发的开源跨平台框架，用于构建多模态应用，如计算机视觉、音频、传感器数据处理等。它特别适用于实时多媒体应用的开发，提供了一系列的工具和预构建的模块，使得开发者能够快速实现复杂的机器学习管道。

1. **模块化设计**：MediaPipe 采用模块化设计，允许开发者通过组合不同的处理模块来构建管道。每个模块可以独立开发和优化。
2. **跨平台支持**：MediaPipe 支持多种平台，包括 iOS、Android、Windows、Linux 和 macOS，使得开发者能够为不同的设备和操作系统构建应用。
3. **实时性能**：框架优化了性能，能够实现实时的数据处理和机器学习推理。
4. **多模态数据处理**：MediaPipe 不仅限于视觉数据，它还支持音频和传感器数据的处理，使得它可以用于多种类型的应用。
5. **易于集成**：MediaPipe 提供了简单的 API，使得开发者可以轻松地将机器学习模型集成到他们的应用中。

6. **预构建解决方案:** MediaPipe 提供了一系列的预构建解决方案, 如面部识别、手势识别、姿势估计等, 这些可以直接用于开发或作为参考。
7. **可扩展性:** 开发者可以根据自己的需求扩展 MediaPipe 的功能, 添加新的模块或改进现有模块。
8. **社区支持:** 作为一个开源项目, MediaPipe 拥有活跃的社区, 提供支持和贡献代码。
9. **文档和示例:** MediaPipe 提供了详细的文档和示例代码, 帮助开发者快速上手。
10. **机器学习模型集成:** MediaPipe 支持多种机器学习模型, 包括 TensorFlow 和 TensorFlow Lite, 使得它可以用于深度学习应用。

1.5.4. UDP (用户数据报协议, User Datagram Protocol)



一种无连接的网络传输协议, 它属于传输层协议, 与 TCP (传输控制协议) 并列, 但两者在设计 and 用途上有所不同。

UDP 协议的主要特点如下:

1. **无连接:** UDP 是无连接的协议, 发送数据之前不需要建立连接。它只是简单地将数据打包到数据报中并发送, 不保证数据报的顺序或可靠性。
2. **简单性:** UDP 协议的实现相对简单, 因为它没有复杂的控制机制, 如 TCP 中的流量控制、拥塞控制和重传机制。

3. **轻量级：** 由于缺乏复杂的控制机制，UDP 的头部开销较小（只有 8 字节），这使得它在传输少量数据时非常高效。
4. **快速性：** UDP 协议的传输速度快，因为它不会等待确认或重传丢失的数据包，适用于对实时性要求较高的应用。
5. **不保证可靠传输：** UDP 不保证数据包的可靠传输，不会对丢失的数据包进行重传，也不会保证数据包的顺序。
6. **数据报：** UDP 使用数据报作为传输单元，每个数据报都是独立的，包含目标端口号和源端口号，以及校验和用于错误检测。
7. **广播和多播：** UDP 支持广播和多播传输，这使得它适用于需要向多个接收者发送相同数据的应用。
8. **应用场景：** UDP 常用于那些可以容忍一定数据丢失，但对传输速度和时延要求较高的应用，如实时视频会议、在线游戏、DNS 查询等。
9. **校验和：** UDP 提供了一个简单的错误检测机制，即校验和，用于检测数据在传输过程中是否出现错误。
10. **端口号：** UDP 使用端口号来区分不同的服务或进程，确保数据能够正确地分发到目标应用程序。

UDP 在机器人中的应用：

1. **实时控制：** 机器人控制通常要求实时响应，UDP 由于其低延迟特性，非常适合实时控制系统。
2. **传感器数据传输：** 机器人上的传感器产生的数据可以通过 UDP 快速发送到控制单元进行处理。

3. **控制命令下发:** 控制中心或操作员发出的控制命令可以通过 UDP 快速发送到机器人, 以实现即时控制。
4. **多机器人协调:** 在多机器人系统中, UDP 可以用于机器人之间的通信, 实现协调动作和数据共享。
5. **简化的通信架构:** UDP 的无连接特性简化了通信架构, 减少了建立和维护连接的开销。
6. **容错机制:** 虽然 UDP 不保证数据的可靠传输, 但在机器人控制中, 可以通过应用层的设计来实现必要的容错机制, 如通过时间戳和序列号来重构数据流。
7. **网络配置灵活性:** UDP 支持广播和多播, 使得在配置网络时更加灵活, 可以方便地实现一对多或多对多的通信。
8. **简化的开发:** UDP 协议的简单性使得开发者可以快速开发和部署通信代码, 加速机器人系统的开发周期。
9. **与 TCP 的结合使用:** 在某些情况下, UDP 可以与 TCP 协议结合使用, 利用 UDP 的快速性和 TCP 的可靠性来满足不同的通信需求。
10. **远程控制:** 在需要远程操作机器人的场景中, UDP 可以用于快速传输操作员的输入到机器人端。
11. **状态更新:** 机器人的状态信息 (如位置、速度、电池电量等) 可以通过 UDP 定期发送到监控系统。
12. **紧急停止信号:** 在紧急情况下, UDP 可以用来快速发送停止信号到机器人, 以确保安全。

1.5.5. 手掌特征检测

是计算机视觉和人机交互领域的一个重要研究方向。这项技术能够使计算机识别和理解手掌的位置、姿势和运动，进而实现对机器人、虚拟角色或其他交互系统的控制。

研究背景

1. **人机交互：**随着技术的发展，人们越来越追求自然和直观的人机交互方式。手掌作为人类最灵活的部位之一，提供了丰富的交互潜力。
2. **虚拟现实和增强现实：**在 VR 和 AR 应用中，手掌跟踪可以提供一种无需手持控制器的交互方式，增强沉浸感。
3. **远程操作和控制：**在远程操作机器人或机械臂时，手掌跟踪可以作为控制信号的来源，提高操作的灵活性和精确性。
4. **健康监测和康复：**手掌关键点检测可以用于监测和分析手部运动，帮助医生评估和治疗肌肉骨骼疾病或康复训练。
5. **娱乐和游戏：**在电子游戏和互动娱乐中，手掌跟踪可以提供一种新的游戏玩法，增加用户体验的丰富性。

原理

1. **图像采集：**首先，通过摄像头或其他视觉传感器采集手掌的图像。
2. **预处理：**对采集到的图像进行预处理，包括灰度化、滤波、降噪等，以提高后续处理的准确性。
3. **手掌检测：**使用肤色模型、轮廓检测、区域生长等方法从图像中检测出手部区域。
4. **特征提取：**提取手掌区域的特征，如纹理、形状、边缘等。
5. **关键点定位：**使用机器学习或深度学习方法训练模型，以识别手掌上的关键点，如手腕、掌根、指尖等。

6. **手势识别**：根据关键点的位置和运动，识别特定的手势或动作。
7. **实时跟踪**：在视频流中实时跟踪手掌关键点，以实现连续的交互。

技术挑战

1. **光照变化**：不同光照条件下，手掌的外观会发生变化，影响检测的准确性。
2. **遮挡问题**：手掌或手指的相互遮挡会使得关键点检测变得困难。
3. **手部姿势变化**：手掌可以做出多种复杂的姿势，模型需要能够准确识别不同的姿势。
4. **实时性要求**：实时交互系统要求关键点检测算法具有较低的延迟。
5. **个性化差异**：不同人的手掌大小、形状和肤色都有所不同，算法需要具有较好的泛化能力。
6. **复杂背景**：在复杂背景中准确检测手掌也是一个挑战。

1.5.6. 特征点姿态解算 (Feature Point Pose Estimation)

特征点姿态解算是计算机视觉和机器人学中的一个关键技术，它涉及到确定图像中特征点的位置以及它们在三维空间中的姿态（位置和方向）。这项技术在多个领域有着广泛的应用，包括机器人导航、增强现实、三维重建、动作识别等。

基本概念

1. **特征点 (Feature Points)**：在图像中具有明显特征的点，如角点、边缘点或其他显著的纹理点。
2. **姿态 (Pose)**：指物体在空间中的位置（位置坐标）和方向（旋转角度）。
3. **二维图像坐标**：特征点在图像平面上的坐标，通常表示为 (x, y) 。
4. **三维空间坐标**：特征点在现实世界或虚拟三维空间中的坐标，通常表示为 (X, Y, Z) 。
5. **相机模型**：用于将三维空间坐标转换为二维图像坐标的数学模型，如针孔相机模型。

姿态解算步骤

1. **特征点检测**: 在图像中检测出特征点, 常用的算法有 Harris 角点检测、SIFT、SURF 等。
2. **特征描述**: 为每个特征点生成描述符, 描述符能够代表特征点的特性, 且对图像的旋转、缩放和部分亮度变化具有不变性。
3. **特征匹配**: 在多张图像或图像与三维模型之间匹配特征点, 常用的匹配算法有最近邻搜索、FLANN 等。
4. **相机标定**: 确定相机的内参 (焦距、主点坐标) 和外参 (相机的位置和旋转), 通常通过标定板来完成。
5. **对极几何**: 利用匹配的特征点对和相机参数, 通过对极几何约束求解特征点在三维空间中的坐标。
6. **三角测量**: 根据至少两个视图中的匹配点对和相机参数, 使用三角测量原理计算特征点的三维坐标。
7. **姿态估计**: 结合三维坐标和二维图像坐标, 估计物体的姿态, 这可能涉及到复杂的数学优化算法。
8. **鲁棒性**: 指系统、模型或算法在面对错误输入、异常值、噪声干扰或不确定性条件下, 仍能保持稳定性和有效性的能力。在不同的领域, 鲁棒性的具体含义可能有所不同, 但总体上, 它强调的是系统对各种干扰因素的抵抗能力和适应能力。
9. **鲁棒性增强**: 在实际应用中, 可能还需要考虑噪声、遮挡和光照变化等因素, 使用 RANSAC 等算法来提高解算的鲁棒性。

(相机标定) 的意义:

- a. 相机镜头通常会产生畸变, 如桶形畸变或枕形畸变。标定可以帮助识别和校正这些畸变, 提高图像质量。
- b. 准确的相机模型可以增强特征匹配算法的性能, 因为它能更准确地预测特征点在图像中的位置。

- c. 标定过程提供了将三维世界坐标映射到二维图像坐标所需的精确参数，反之亦然。

应用领域：

- **机器人导航**: 通过特征点姿态解算，机器人可以确定自身在环境中的位置和方向。
- **增强现实**: 在 AR 中，特征点姿态解算用于将虚拟对象准确地叠加到现实世界中。
- **动作识别**: 通过分析人体关节的姿态变化，可以识别和理解人体动作。
- **三维重建**: 特征点的姿态信息是三维重建过程中的关键，用于恢复场景的三维结构。

1.6. 实验道具讲解

当机器狗通过摄像头识别到完整手掌后，随着人的手掌的移动，机器狗会朝手掌移动的方向行走，即手掌跟踪。

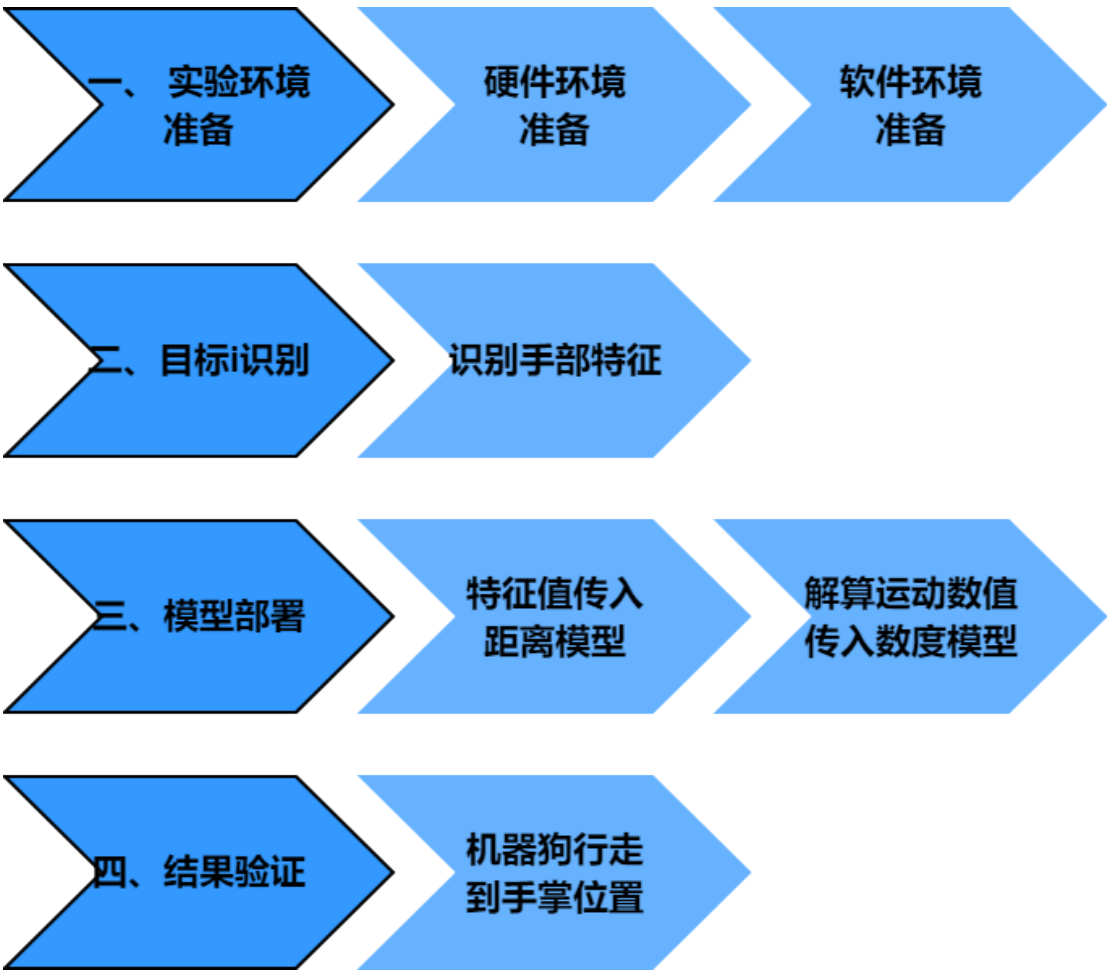


第二章 实验过程简述

本项目的主要流程为：

通过 MediaPipe 手部检测模型→特征值传入距离模型，确定手部的距离和位置→根据手部与摄像头的距离和位置计算运动数值→根据计算出的速度和方向，控制机器狗行走至手掌位置

本实验分成如下步骤完成：



第三章 实验环境准备

3.1. 硬件环境准备

(1) SD 卡中系统烧录

向 SD 卡中烧录镜像的步骤如下。

① 将 SD 卡插入读卡器，再将读卡器插到 PC 上。



② 在浏览器中进入 [Ascend 官网](https://ascend.com.cn)，选择“产品”->“开发者套件”



③ 点击“资源”，进入资源下载界面



④ 选择“制卡工具下载”，下载完成后进行安装。



⑤ 打开镜像烧录工具 Ascend AI Devkit Imager，选择 Atlas 200I DK A2



⑥打开“配置网络信息”，选择“Type-C”选项卡。可以看到 Type-C 接口默认 IP 为 192.168.0.2。选择默认 IP。

镜像网络信息配置

以下为镜像默认IP信息，您可以根据PC/路由器IP信息重新填写网络信息

网络信息

ETH0

ETH1

Type-C

☐ 自动获取IP地址

☒ 使用下面的IP地址

IP地址

192 · 168 · 0 · 2

子网掩码

255 · 255 · 255 · 0

默认网关

· · ·

首选DNS服务器

8 · 8 · 8 · 8

备用DNS服务器

114 · 114 · 114 · 114

确定

取消

⑦选择要烧录的 SD 卡

昇腾开发者套件一键制卡工具

1 选择操作

在线制卡

本地制卡

备份SD卡

镜像版本

Desktop-Image_23.0.RC3_ubuntu22.04

Atlas 200I DK A2

配置网络信息

镜像包含OS、NPU驱动固件、CANN、MindX SDK、图形桌面、应用桌面、USB Audio...

OS版本: Ubuntu 22.04 LTS Arm64

固件与驱动版本: 23.0.RC3

CANN版本: 7.0.RC1

发布时间: 2023-11-15

缓存到本地磁盘

镜像版本

Base-Image_23.0.RC1_ubuntu18.04

Atlas 200 DK

Atlas 200DK镜像, 预装Python、CANN、MindX SDK、SSH、SAMBA、VNC、XRD...

OS版本: Ubuntu 18.04 LTS Arm64

固件与驱动版本: 1.0.13.alpha

CANN版本: 5.1.RC2.alpha007

发布时间: 2022-11-28

需要下载: 8.7 GB

2 选择SD卡

SD卡大小: 62.5 GB

Mass Storage Device USB Device

Mass Storage Device USB Device-62.5 GB

盘符: F:\, H:\, I\

Kingston DataTraveler 3.0 USB Device-31.0 GB

盘符: G\

3 烧录镜像

点击按钮烧录镜像

烧录镜像

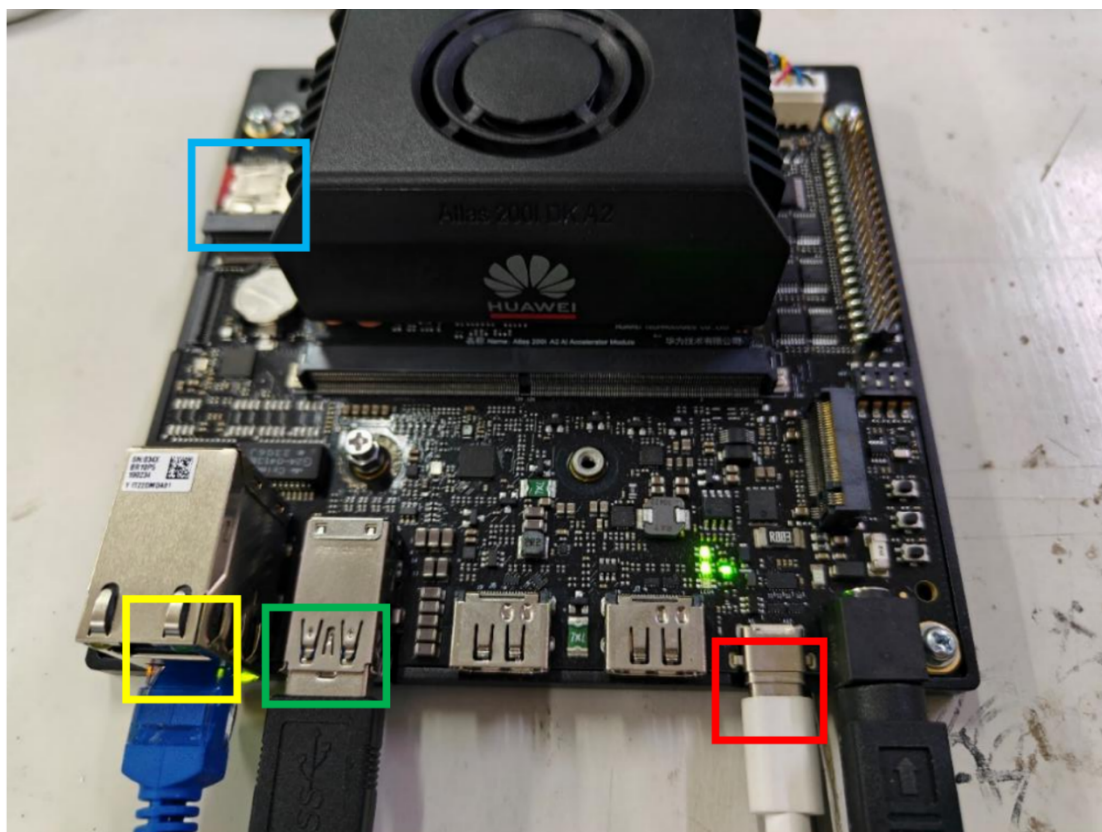
⑧点击“烧录镜像”，等待镜像烧录完成。

Error! Unknown document property name.



(2) 开发板连接

将烧录好系统的 SD 卡插入到开发板中（下方蓝色框中），昇腾开发板和 PC 机通过 Type-C 线连接，将 Type-C 线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的 USB 线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



(3) PC 端远程连接开发板

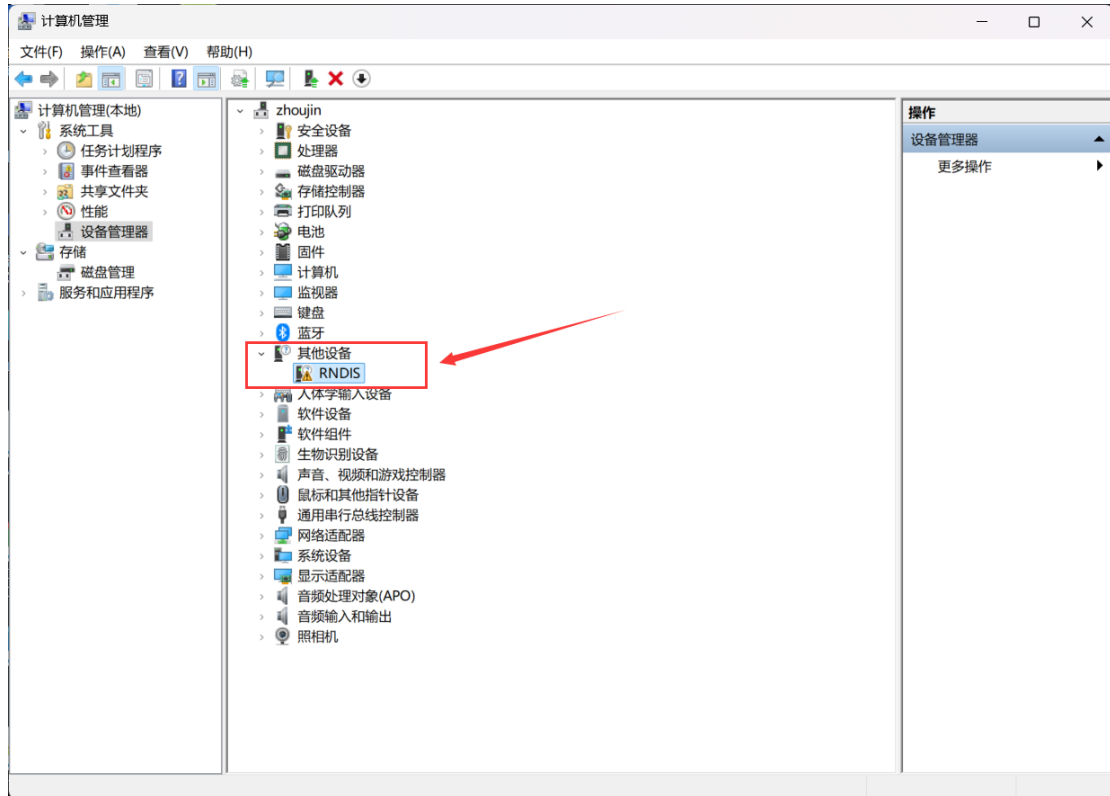
本节介绍当开发者套件通过 Type-C 接口和 PC 连接时,如何在 PC 将接口 IP 地址修改为和开发者套件接口 IP 地址同一个网段 (如开发者套件 Type-C 接口为 192.168.0.2, PC 对应 USB 或 Type-C 接口为 192.168.0.101), 并使用 SSH 工具远程登录开发者套件。

1) 安装 Windows 的 USB 网卡驱动

本步骤以 Windows 10 系统为例。

①在 PC 上右键单击“此电脑”, 选择 “更多” → “管理”, 进入“计算机管理”界面。

②在“计算机管理”操作界面中选择“设备管理器→ “其他设备”, 如图所示, RNDIS 为未识别状态。

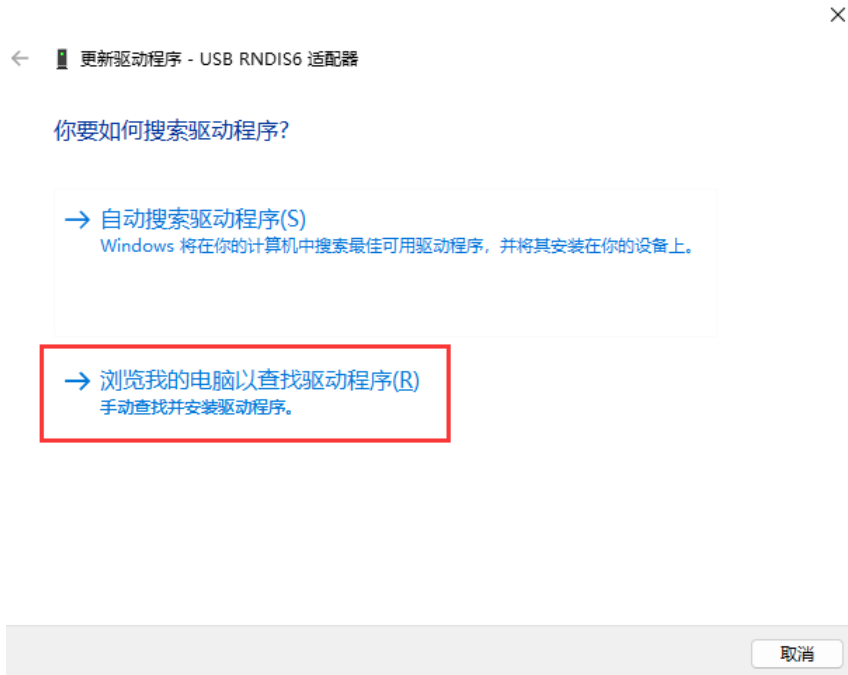


注意：如果未出现 RNDIS，请按以下方法排查：

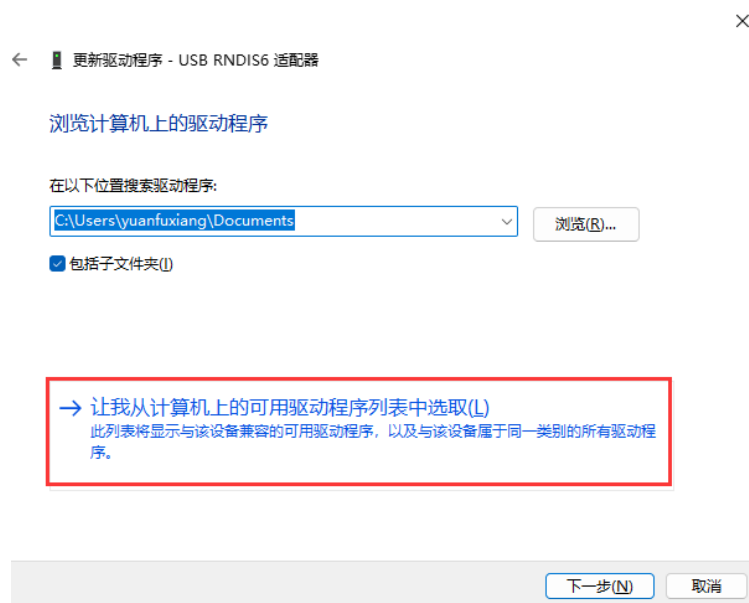
- a) 确保开发者套件 Type-C 接口和 PC 已通过连接线正常连接。
- b) 更换具备数据传输能力的 Type-C 数据线（一般手机充电线拥有数据传输能力），继续查看是否出现 RNDIS。
- c) 如果尝试现场所有 Type-C 数据线，仍无法出现 RNDIS，则考虑使用 RJ45 网线连接 PC 和开发者套件的以太网口的方式登录开发者套件。

③右键单击“RNDIS”，选择“更新驱动程序(P)”。

④在弹出的“更新驱动程序”→“RNDIS 窗口”中选择“浏览我的计算机以查找驱动程序软件 (R)”。



⑤然后选择“让我从计算机上的可用驱动程序列表中选择(L)”。



⑥在“常见硬件类型”列表中选择“网络适配器”，单击“下一页(N)”。

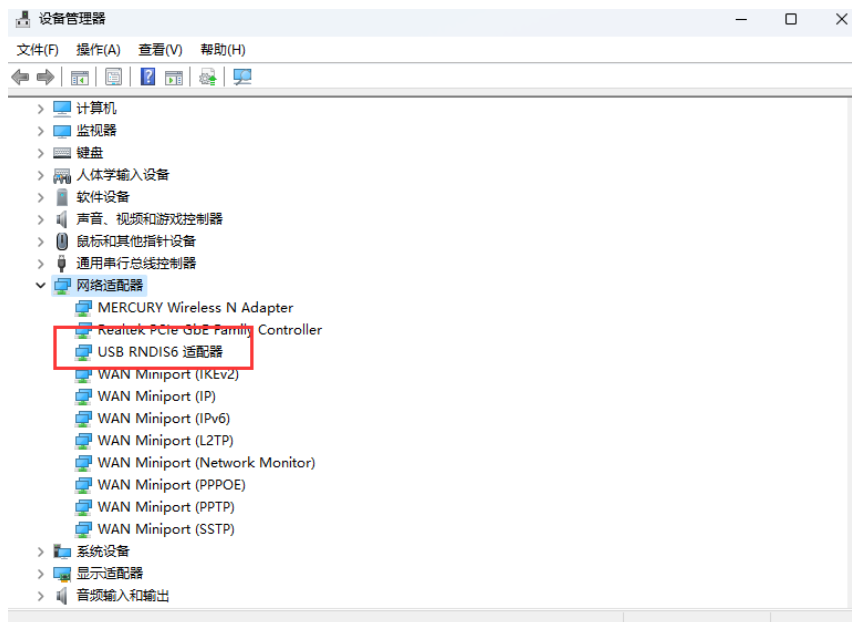


⑦在“选择要为此硬件安装的设备驱动程序”界面中选择“Microsoft”厂商的“USB RNDIS6 适配器”。



⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。

⑨返回“设备管理器”→“网络适配器”，可看到已经正常显示了 USB RNDIS6 适配器的驱动。



2) 配置 PC 接口 IP 地址

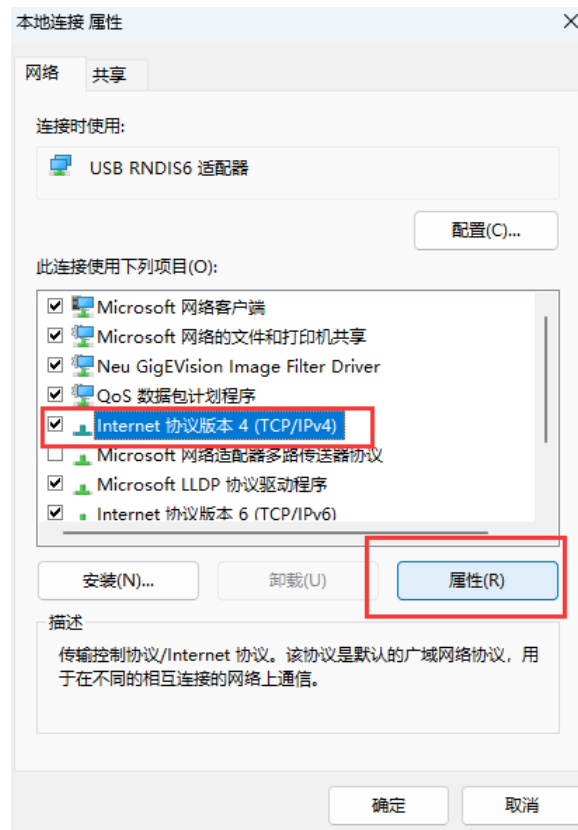
本步骤以 Windows 10 系统为例。

①在 PC 上打开“开始”，单击“设置”按钮，进入“Windows 设置”界面。

②选择“网络和 Internet”，单击“更改适配器选项”。

③鼠标右键单击“本地连接”后鼠标左键单击“属性”进入“本地连接 属性”界面（使用 Type-C 接口连接时一般为“本地连接 x”，x 为数字，以实际显示的数字为准）。

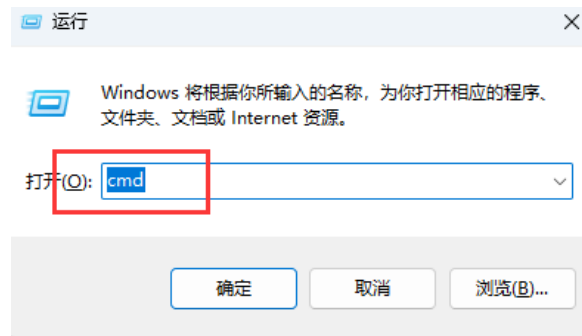
④选择“Internet 协议版本 4 (TCP/IPv4)”，单击“属性”。



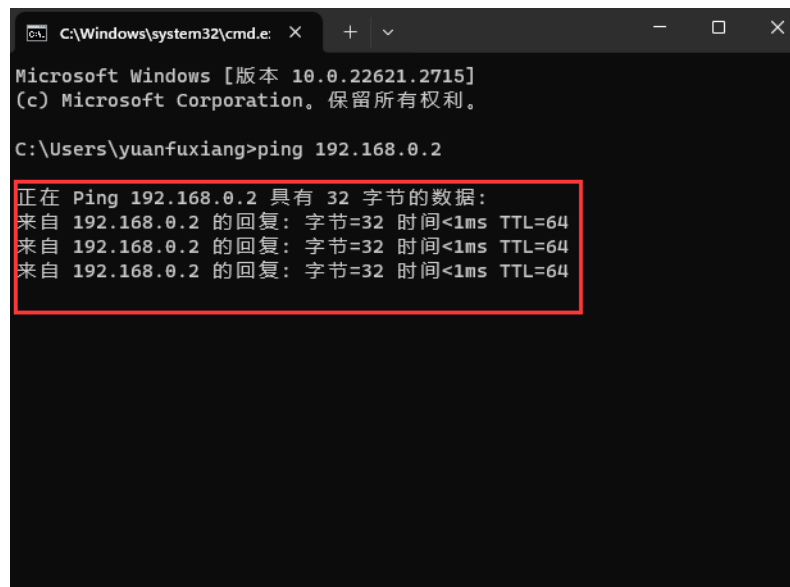
⑤勾选“使用下面的 IP 地址”选项，填写 IP 地址（图示以 192.168.0.101 为例）和子网掩码，默认网关与 DNS 服务器地址为空，单击“确定”保存。



⑥使用快捷键“Win+R”，在运行窗口输入 cmd 进入命令行窗口。



⑦输入 ping 192.168.0.2 命令，查看是否能够访问开发板 IP。

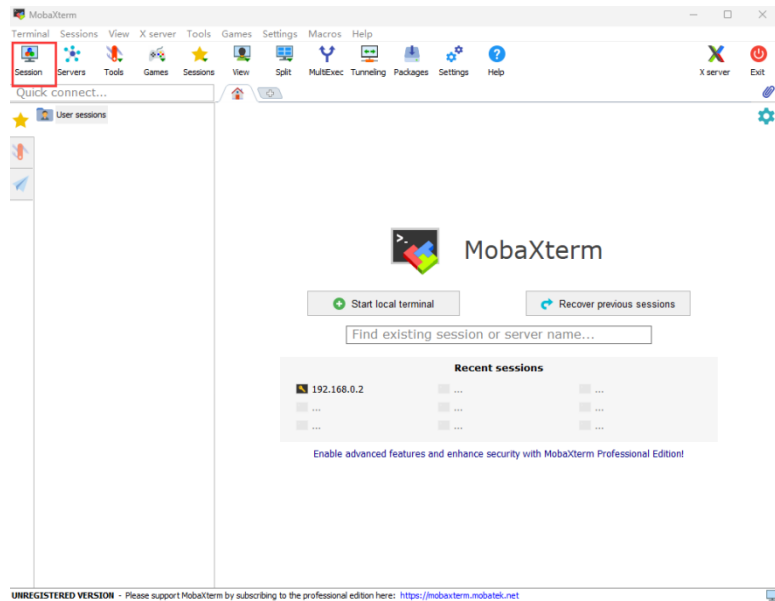


3) 使用 ssh 远程登录开发板

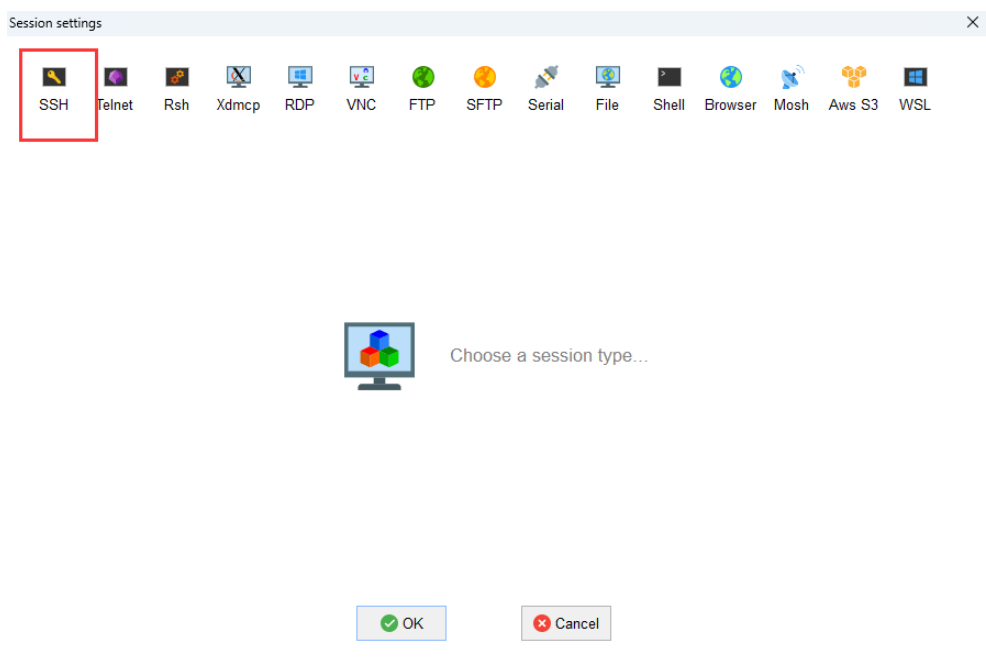
本文使用 MobaXterm 工具在 PC 上远程登录开发板，单击[下载链接](#)获取 MobaXterm 软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

①双击 “MobaXterm_Personal_22.2.exe” 启动 SSH 登录工具。

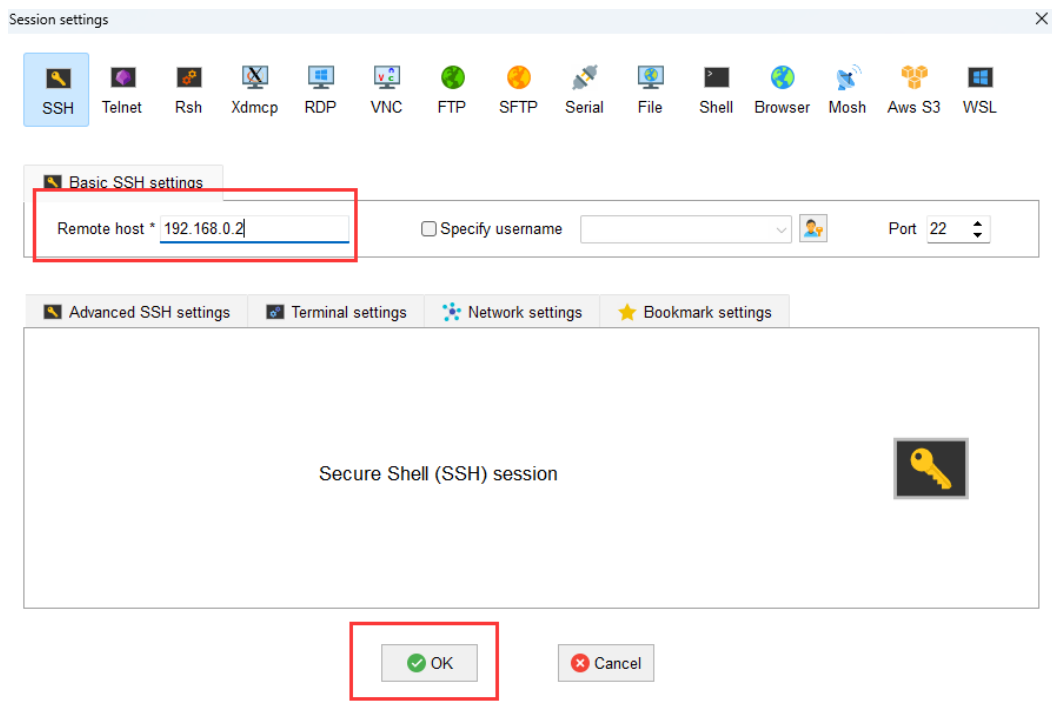
②单击左上方的 “Session” 进入界面。



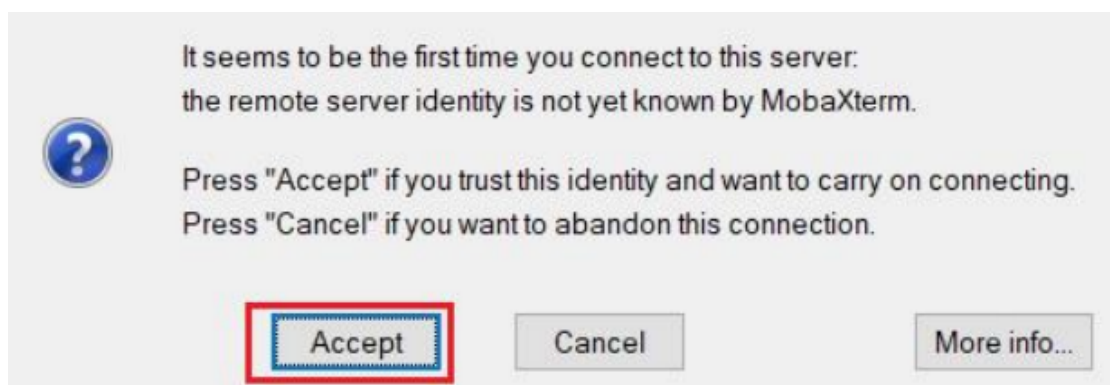
③单击左上方的“SSH”进入 SSH 连接配置界面



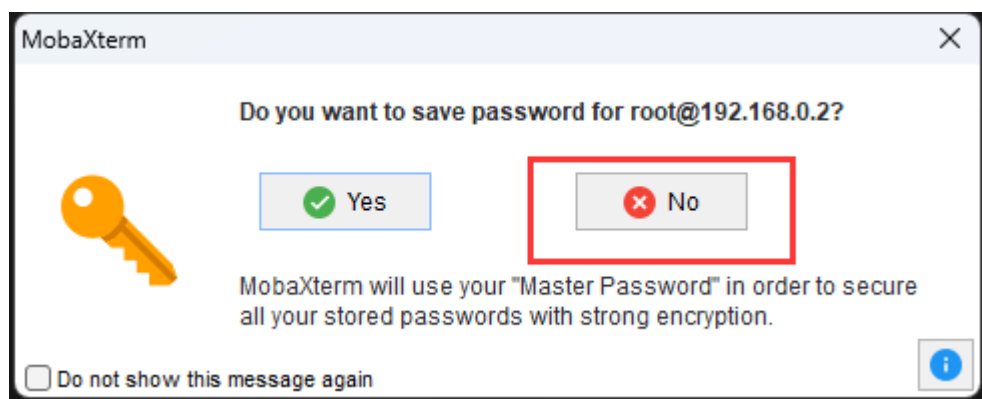
④根据硬件连接方式填写开发者套件连接 PC 的接口实际 IP 地址（一键制卡中配置的 IP 地址）。图示以 192.168.0.2 为例，请根据实际修改。



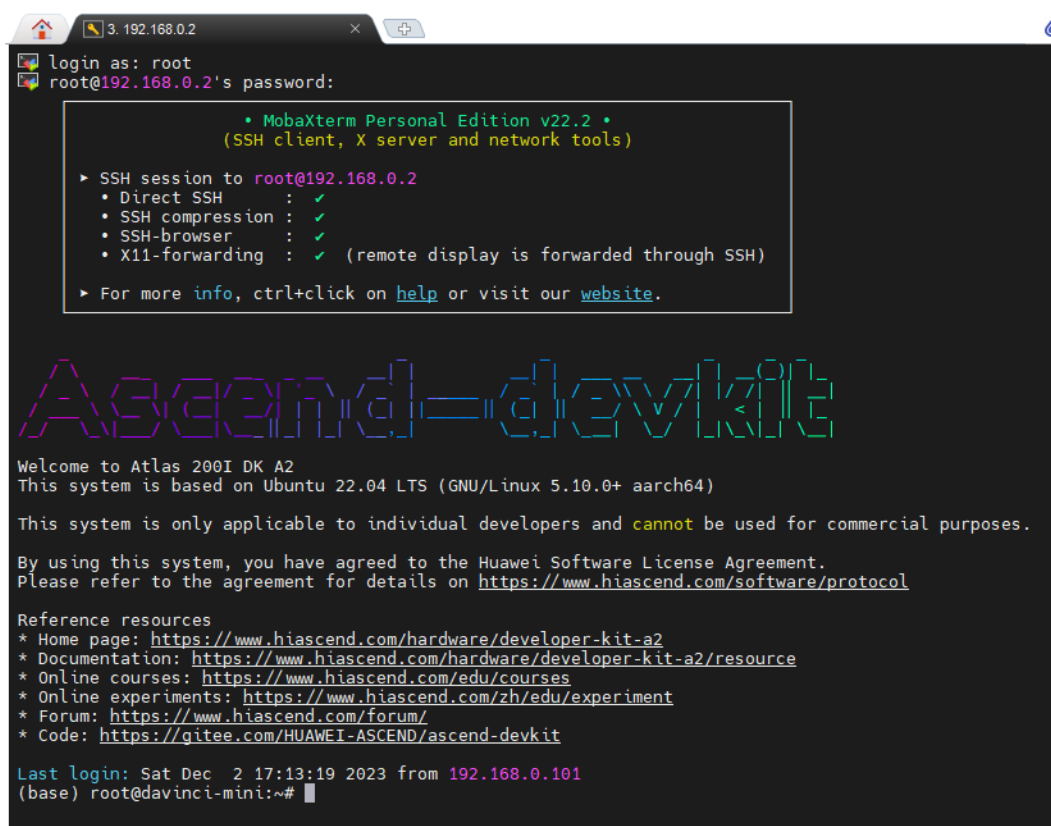
⑤单击“OK”按钮，首次连接开发者套件时，SSH 工具提示是否信任连接的设备，单击 “Accept”。



⑥进入远程登录界面后，输入 root 用户名登录密码（默认为 Mind@123）登录开发者 套件，请修改默认密码，并妥善保管新密码。 SSH 工具界面会出现保存密码提示，可以单击“No”，不保存密码直接登录开发者套件。



⑦远程登录开发者套件成功界面如图所示。



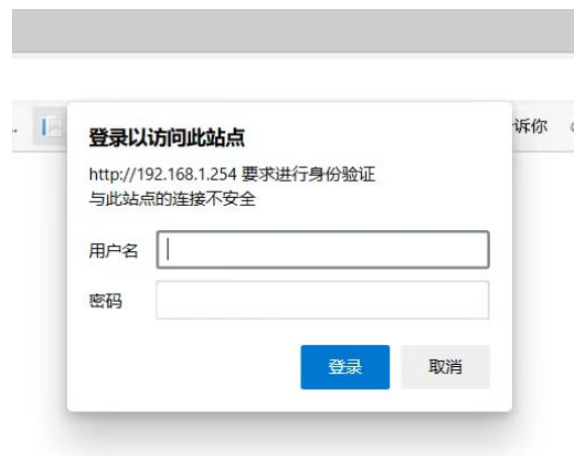
3) 机器狗 中继连接外网

注: 本教程主要为暂时没有外置无线网卡的用户提供连接外网方案, 教程以中继手机热点为例进行说明, 中继无线路由器同理。

1、电脑连接机器狗 的 wifi, 点击电脑右下角的 wifi 图标, 点击属性查看 IPv4 DNS 服务器的 ip 地址。如下图所示。



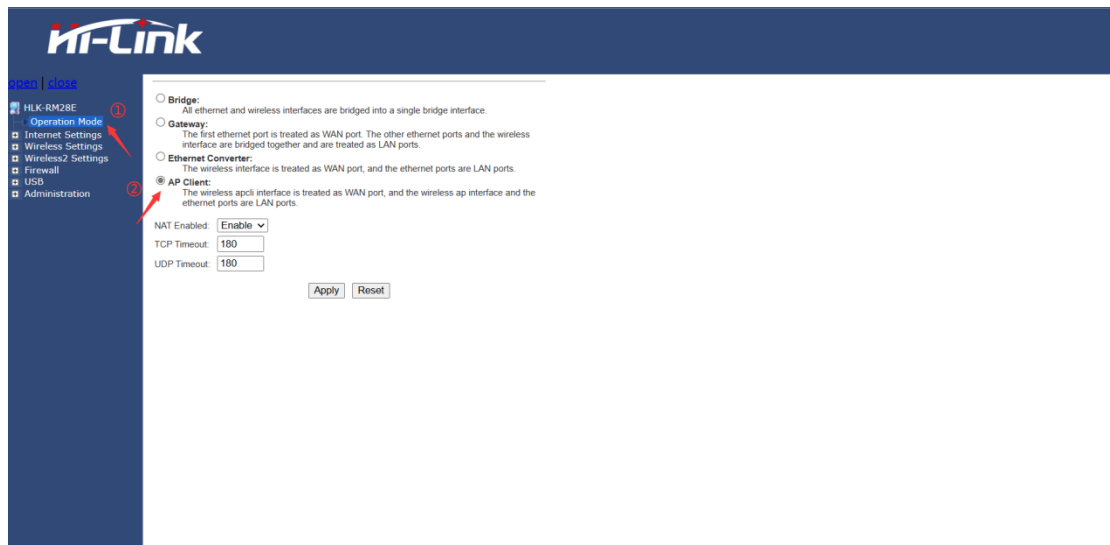
2、打开电脑浏览器，输入 IPv4 DNS 服务器的 ip，默认登录用户和密码均为 admin。如下图所示。



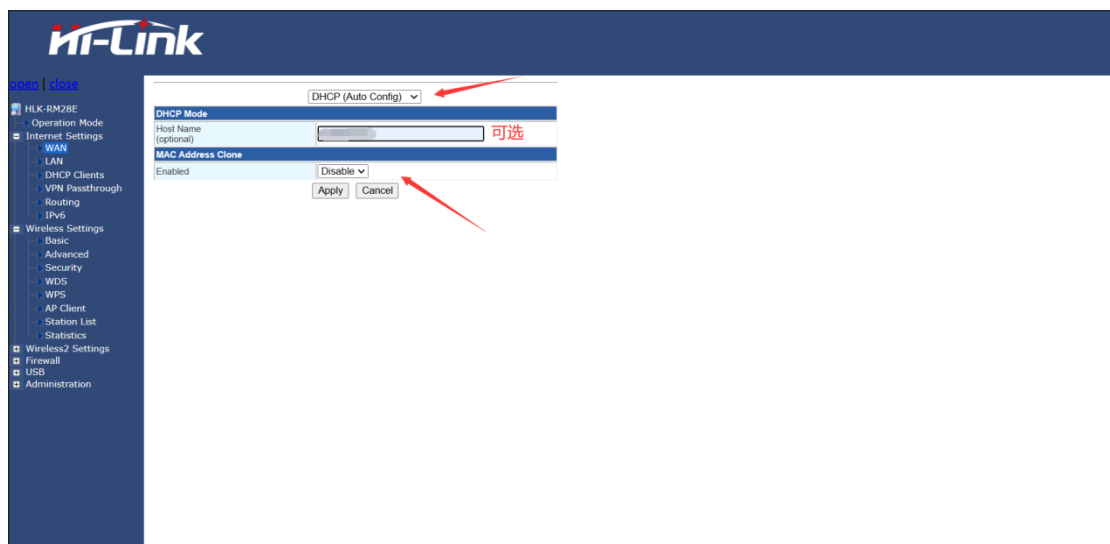
3、打开手机热点。

4、进入 Hi-link 设置页面，点击 Operation Mode，选择 AP Client，最后点击

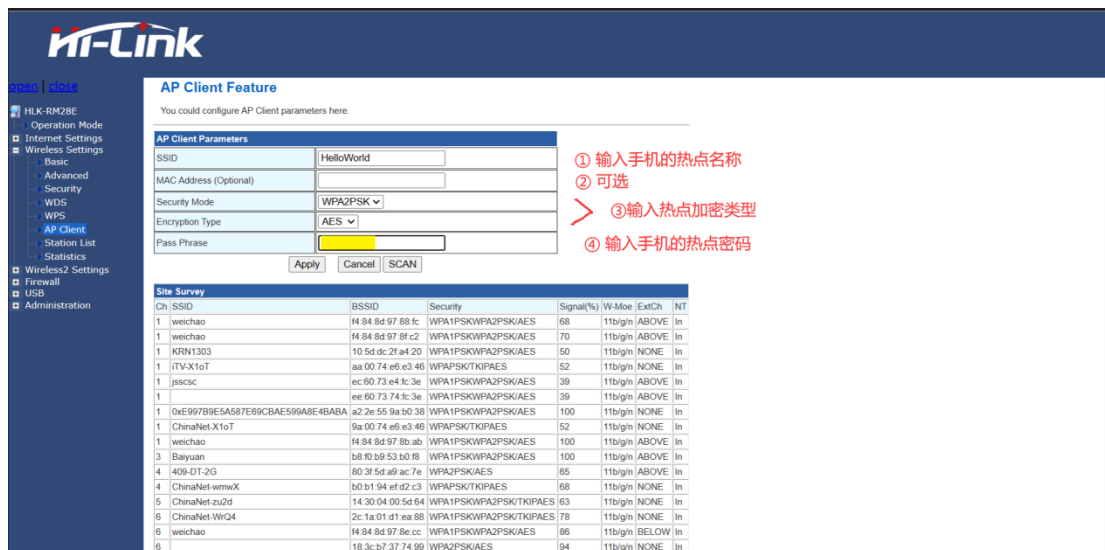
Apply。如下图所示。



5、展开 Internet Settings，点击 WAN，参照下图进行设置，最后点击 Apply。



6、展开 Wireless Settings，点击 AP Client，查看 site Survey 表中是否有自己的手机热点名称，如若没有点击 SCAN 进行扫描，在 AP client Parameters 表中参照下图进行设置，最后点击 Apply。



7、重启机器狗,注意中继时热点需一直处于开启状态,电脑连接机器狗的wifi,使用远程软件登录,此时机器狗处于连接外网状态。测试是否有网络,点击浏览器,搜索任意内容,如“百度学术”,能正常显示下图说明正常。



3.2. 软件环境准备

安装 cvzone 相关的库

1. `pip install cvzone`

如果出现其他的包缺失,按照报错提示安装即可

3.3. 机器狗远程连接

用户可通过 SSH 远程连接到运动主机。

1. 将开发主机连接到机器人 WiFi。
2. 在开发主机上打开 SSH 连接软件，输入 `ssh firefly@192.168.1.120`，密码为 `firefly`，即可远程连接运动主机；
3. 进入 `jy_exe`，并打开配置文件：

```
cd /home/firefly/jy_exe/conf/  
vim network.toml
```

4. 配置文件 `network.toml` 内容如下：

```
ip = '192.168.1.102'  
  
target_port = 43897  
  
local_port = 43893  
  
~
```

5. 修改配置文件第一行中的 IP 地址，使得代码能够接收到机器狗数据：

- 如果代码在机器人运动主机内运行，IP 设置为运动主机 IP：192.168.1.120；
- 如果代码在用户自己的开发主机中运行，设置为开发主机的静态 IP：

192.168.1.xxx；

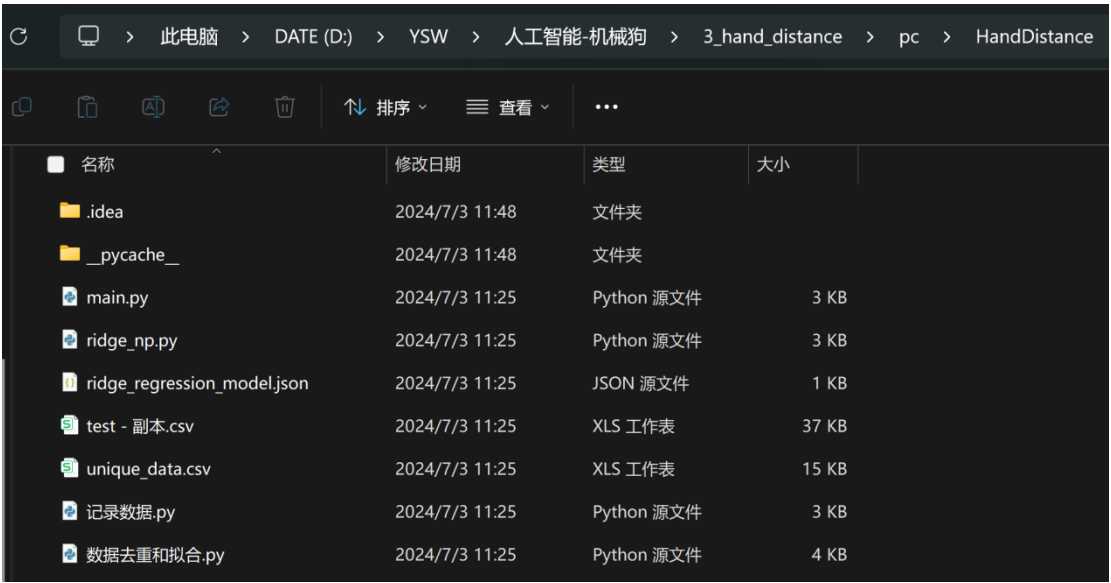
6. 重启运动程序使配置生效。

```
cd /home/firefly/jy_exe  
  
sudo ./stop.sh  
  
sudo ./restart.sh
```

第四章 工程文件目录结构

4.1. PC

这个章节存放的内容主要作用是采集手掌数据，对数据进行采集，处理，建模。



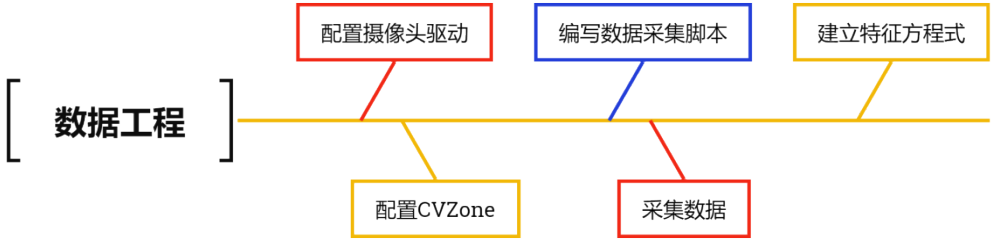
记录数据.py 是采集的脚本，数据保存在了 test-副本.csv 中，数据去重和拟合.py 是处理数据与分析数据的脚本，分析后的数据保存在了 unique_data.csv 中，然后得到的方程保存在了 ridge_regression_model.json, ridge_np.py 是计算的距离的函数。

4.2. Ascend

```
(base) root@davinci-mini:/opt/lab/HandTracking# tree -I "__pycache__|*.pyc" -L 10
.
├── HandDistance
│   ├── main.py
│   ├── ridge_np.py
│   ├── ridge_regression_model.json
│   ├── test - 副本.csv
│   ├── unique_data.csv
│   ├── 数据去重和拟合.py
│   └── 记录数据.py
├── camera
│   ├── HKcamera.py
│   └── det_utils.py
├── command
│   ├── __init__.py
│   └── udp_command.py
├── hand_distance_main.py
├── robotstatuswatcher
│   ├── __init__.py
│   └── listener.py
├── sendcommand
│   ├── SendToCommand.py
│   ├── __init__.py
│   └── heartbeat.py
├── socketnetwork
│   ├── __init__.py
│   └── network_utils.py
├── speeds
│   ├── __init__.py
│   └── sportsspeed.py
├── threading_utils
│   └── ThreadTemplates.py
```

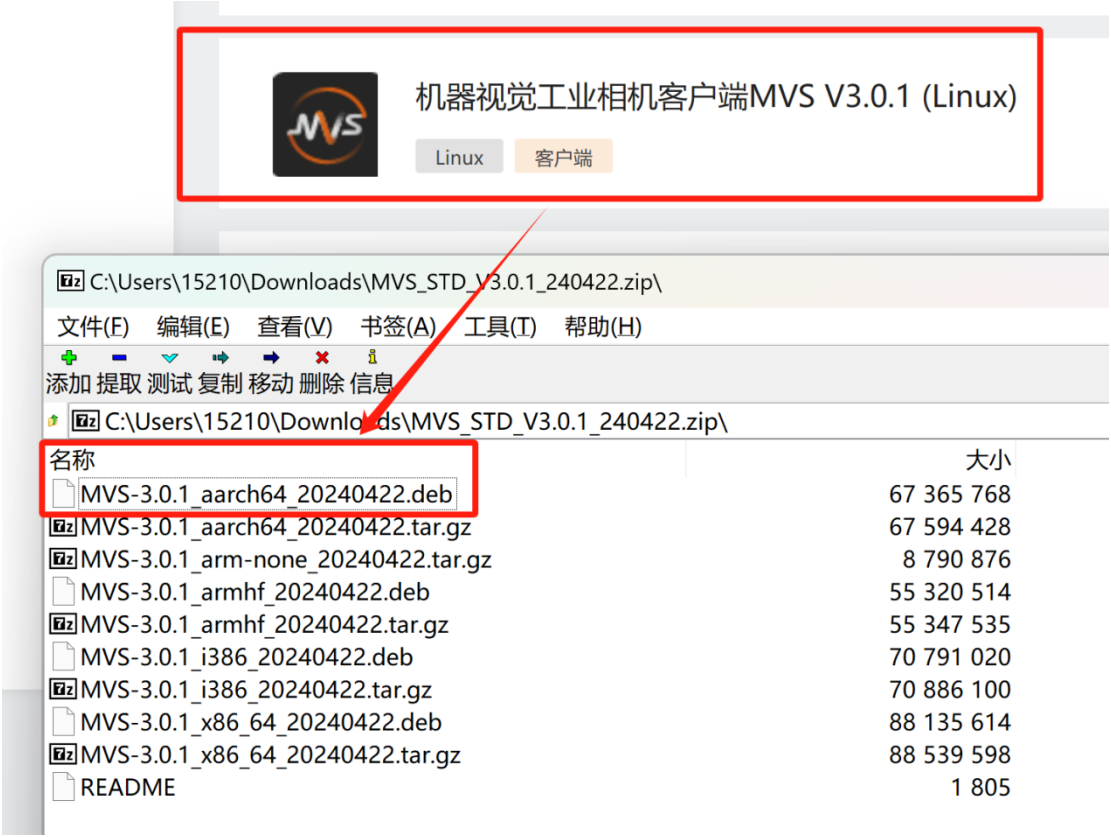
开发板的文件夹总名叫做 ‘HandTracking’，如图所示，其中 command 文件中的 udp_command.py 文件的用来存放指令码，sendcommand 中的 heartbeat.py 是发送心跳包，SendToCommand.py 是发送指令，socketnetwork 中的 network_utils.py 是与机器狗建立通信的脚本，speeds 中的 sportsspeed.py 文件是机器狗运动的速度方程，threading_utils 中的 ThreadTemplates.py 文件是线程，camera 中放的是摄像头的使用文件，robotstatuswatcher 中使用来监听机器狗状态的，HanDistance 中存放的是方程模型与采集数据和分析数据的脚本。

第五章 数据工程



5.1. 配置开发板的 HK 驱动

下载并安装 MVS 驱动文件 (Linux)



海康威视官网下载链接:

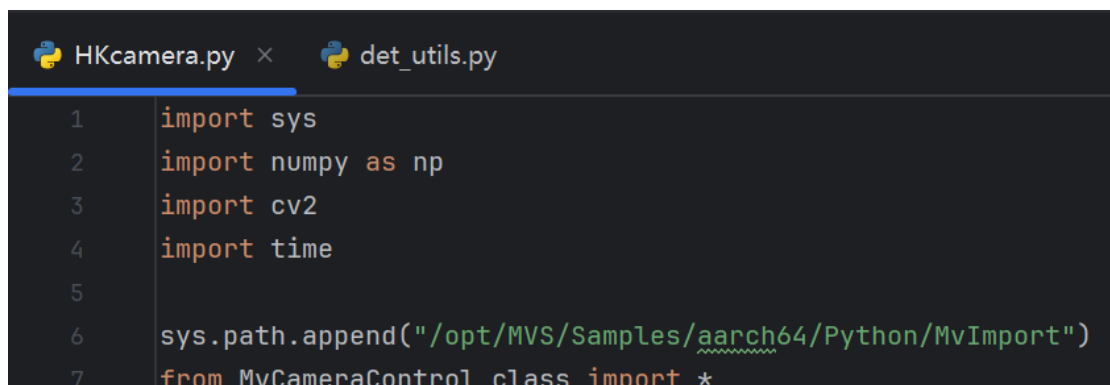
<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>



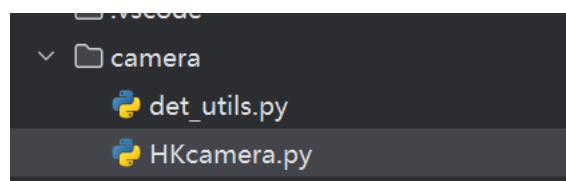
打开 MobaXterm 远程控制，进入开发板安装驱动指令：`dpkg -i MVS-3.0.1_aarch64_20240422.deb`

在代码中使用需要加入包的路径：

```
sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")from  
MvCameraControl_class import*
```



使用摄像头的代码：`python HKcamera.py`



5.2. 配置本地 CVZone 环境

安装 cvzone 相关的库

Error! Unknown document property name.

```
2. pip install cvzone
```

如果出现其他的包缺失，按照报错提示安装即可

5.3. 测试识别效果

调用的测试代码如下

```
3. import math
4.
5. import cv2
6. from cvzone.HandTrackingModule import HandDetector
7. import numpy as np
8. import cvzone
9. import pandas as pd
10. import os
11.
12. cap = cv2.VideoCapture(0)
13. cap.set(3, 640) # 设置帧的宽度
14. cap.set(4, 480) # 设置帧的高度
15.
16. detector = HandDetector(detectionCon=0.8, maxHands=2)
17.
18. while True:
19.     success, img = cap.read()
20.     if success: # 检查帧是否成功读取
21.         # 这个的 draw = False 是指只显示实际距离的检测框，手
部的检测框不显示
22.         hands, img = detector.findHands(img)
23.
```

```

24.         cv2.imshow("Image", img)
25.
26.     # 按 'q' 键退出循环
27.     if cv2.waitKey(1) & 0xFF == ord('q'):
28.         break
29.
30. cap.release()
31. cv2.destroyAllWindows()

```

`import math`: 导入`math`库，用于执行数学运算。

`import cv2`: 导入`OpenCV`库，用于图像和视频处理。

`from cvzone.HandTrackingModule import HandDetector`: 从 `cvzone` 的 `HandTrackingModule`模块中导入`HandDetector`类，用于检测图像中的手部。

`import numpy as np`: 导入`NumPy`库，并简称为`np`，用于高效的数值计算。

`import cvzone`: 导入`cvzone`库，它提供了一些额外的计算机视觉功能。

`import pandas as pd`: 导入`Pandas`库，并简称为`pd`，用于数据处理。

`import os`: 导入操作系统接口库。

`cap = cv2.VideoCapture(0)`: 初始化摄像头，0表示默认的摄像头设备。

`cap.set(3, 640)`: 设置视频捕获的宽度为640像素。

`cap.set(4, 480)`: 设置视频捕获的高度为480像素。

`detector = HandDetector(detectionCon=0.8, maxHands=2)`: 创建`HandDetector`对象，设置检测置信度为0.8，最大检测手部数量为2。

`while True::` 开始一个无限循环，用于持续捕获和处理视频帧。

`success, img = cap.read()`: 从摄像头读取一帧图像，`success`是一个布尔值，表示是否成功读取，`img`是读取的图像本身。

if success:： 如果成功读取图像，则执行以下操作。 19-20. hands, img = detector.findHands(img): 使用HandDetector对象检测图像中的手部，并返回检测到的手部信息和更新后的图像。

cv2.imshow("Image", img): 显示处理后的图像。

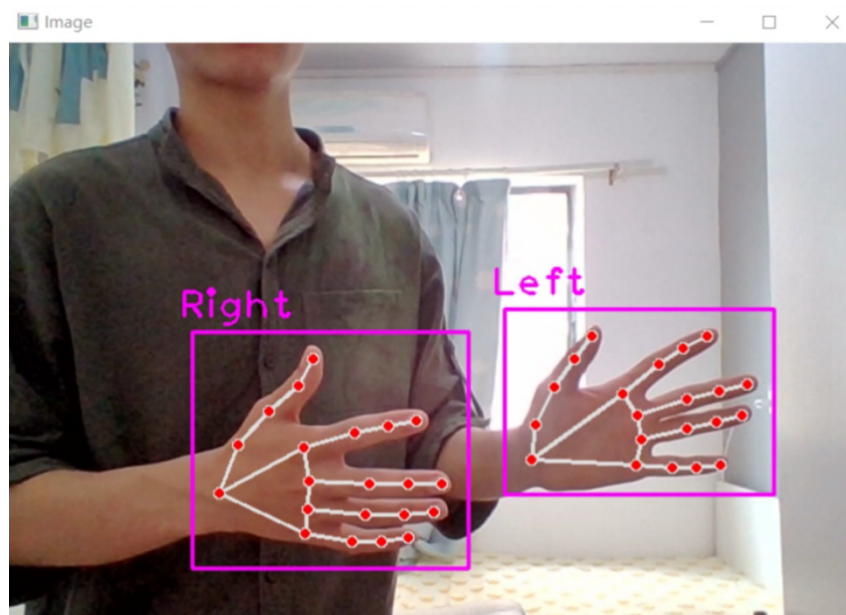
24-26. if cv2.waitKey(1) & 0xFF == ord('q'):： 如果按下'q'键，则退出循环。
cv2.waitKey(1)函数会等待1毫秒的按键事件， & 0xFF用于确保获取按键的准确ASCII值。

break: 退出循环。

cap.release(): 释放摄像头资源。

cv2.destroyAllWindows(): 关闭所有OpenCV创建的窗口。

实际效果如下所示：



5.4. 编写数据采集脚本

我们可以编写一个简单的脚本输出数据看看长什么样子。

```
[398, 332, 0], [402, 312, -21], [416, 294, -34], [434, 295, -46], [
[398, 345, 0], [401, 332, -22], [412, 319, -36], [428, 320, -48], [
[380, 342, 0], [403, 360, -4], [433, 367, -9], [454, 373, -15], [46
[383, 342, 0], [404, 355, -3], [434, 362, -9], [454, 371, -15], [46
[98, 425, 0], [139, 388, -6], [185, 378, -10], [212, 394, -13], [20
Process finished with exit code 0
```

然后编写数据采集的脚本

```
1.  import math
2.
3.  import cv2
4.  from cvzone.HandTrackingModule import HandDetector
5.  import numpy as np
6.  import cvzone
7.  import pandas as pd
8.  import os
9.
10. file_name = 'test.csv'
11.
12.
13.  # 设置CSV文件的列名
14.  columns = ['real_distance', '5-17', '0-12']
15.
16.  # 创建空的DataFrame
17.  data = pd.DataFrame(columns=columns)
18.
19.  # 检查文件是否存在且包含正确的列头
20.  if not os.path.isfile(file_name) or os.path.getsize(file_
name) == 0:
```

```

21.         data.to_csv(file_name, index=False) # 第一次或文件不
        存在, 写入列头
22.     else:
23.         data.to_csv(file_name, mode='a', index=False, header=
        False) # 追加数据, 不包括列头
24.     # 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85,
        90, 95, 100, 105, 110, 120, 130, 140, 150, 200
25.     real_distance = 20
26.     # Hand_direction = 1
27.
28.
29.     cap = cv2.VideoCapture(0)
30.     cap.set(3, 640) # 设置帧的宽度
31.     cap.set(4, 480) # 设置帧的高度
32.
33.
34.     # 实例化手部检测器, 此处仅初始化了检测器, 并没有在代码后面使用它
35.     detector = HandDetector(detectionCon=0.8, maxHands=2)
36.
37.     while True:
38.         success, img = cap.read()
39.
40.         if success: # 检查帧是否成功读取
41.             # 这个的 draw = False 是指只显示实际距离的检测框, 手
            部的检测框不显示
42.             hands, img = detector.findHands(img)
43.             if hands:

```

```

44.         lmList = hands[0]['lmList']
45.         # hand_type=['Left', 'Right']
46.         # if hand_type[Hand_direction] == hands[0]['t
type']:
47.         hand_data = []
48.         x, y, w, h = hands[0]['bbox']
49.         x1, y1, z1 = lmList[5]
50.         x2, y2, z2 = lmList[17]
51.         x3, y3, z3 = lmList[0]
52.         x4, y4, z4 = lmList[12]
53.         distance1 = int(math.sqrt((y2 -
y1) ** 2 + (x2 - x1) ** 2))
54.         distance2 = int(math.sqrt((y4 -
y3) ** 2 + (x4 - x3) ** 2))
55.         hand_data += [real_distance, distance1, dista
nce2]
56.
57.         # 添加当前手部数据到DataFrame
58.         data_length = len(data)
59.         data.loc[data_length] = hand_data
60.
61.
62.         cv2.imshow("Image", img)
63.
64.         # 按 'q' 键退出循环
65.         if cv2.waitKey(1) & 0xFF == ord('q'):
66.             break
67.
68.     cap.release()

```

```
69. cv2.destroyAllWindows()
70.
71. # 在捕获循环结束后, 通过指定 mode='a' 和 header=False 来追加数据
72. data.to_csv(file_name, mode='a', index=False, header=False)
```

导入模块

math: 用于数学运算。

cv2: OpenCV 库, 用于图像和视频处理。

HandDetector: **cvzone** 库中的一个类, 用于手部检测。

numpy: 用于数值计算。

pandas: 用于数据处理和 CSV 文件操作。

os: 用于操作系统功能, 如文件路径操作。

设置 CSV 文件

file_name: 定义了 CSV 文件的名称。

columns: 定义了 CSV 文件的列名, 用于存储数据。

data: 创建了一个空的 **DataFrame**, 用于后续存储数据。

检查和准备 CSV 文件

检查 **test.csv** 文件是否存在, 如果不存在或为空, 则创建文件并写入列头。

如果文件已存在且有内容, 则追加数据, 不写入列头。

初始化摄像头

使用 **cv2.VideoCapture(0)** 初始化摄像头, 0 表示默认摄像头。

设置摄像头捕获的帧宽度和高度。

实例化手部检测器

创建 `HandDetector` 对象，设置检测置信度和最大手部数量。

视频捕获循环

使用 `while True` 循环持续捕获视频帧。

使用 `cap.read()` 读取每一帧。

使用 `detector.findHands()` 检测每一帧中的手部。

手部数据处理

如果检测到手部，计算手部关键点之间的距离。

`lmList`：包含手部关键点的列表。

`bbox`：包含手部边界框的坐标。

`distance1` 和 `distance2`：计算得到的两段关键点间的距离。

将实际距离和计算得到的距离作为数据添加到 `DataFrame`。

显示和退出机制

使用 `cv2.imshow()` 显示视频帧。

按下 'q' 键退出循环。

资源释放和数据保存

循环结束后，释放摄像头资源，关闭所有 `OpenCV` 窗口。

将捕获的数据追加到 `CSV` 文件中。

随后，我们可以改变 `real_distance` 的值，得到在不同距离下手部特征值，部分如下所示：

```
test - 副本.csv X
HandDistance > test - 副本.csv
1 real_distance,5_17,0_12
2 25,138,348
3 25,136,358
4 25,138,357
5 25,137,359
6 25,136,358
7 25,137,359
8 25,136,359
9 25,137,361
10 25,137,358
11 25,138,359
12 25,136,357
13 25,136,360
14 25,136,355
15 25,135,356
16 25,135,355
17 25,136,357
18 25,134,357
19 25,136,363
20 25,135,363
21 25,135,363
22 25,135,363
23 25,135,363
```

5.5. 采集手部特征值与数据格式化

采集之后的数据，如下所示。

```
test - 副本.csv X
HandDistance > test - 副本.csv
1 real_distance,5_17,0_12
2 25,138,348
3 25,136,358
4 25,138,357
5 25,137,359
6 25,136,358
7 25,137,359
8 25,136,359
9 25,137,361
10 25,137,358
11 25,138,359
12 25,136,357
13 25,136,360
14 25,136,355
15 25,135,356
16 25,135,355
17 25,136,357
18 25,134,357
19 25,136,363
20 25,135,363
21 25,135,363
22 25,135,363
23 25,135,363
24 25,135,364
25 25,134,362
26 25,135,362
27 25,135,364
28 25,136,364
29 25,135,363
30 25,134,362
31 25,135,364
32 25,135,365
33 25,135,364
34 25,136,365
35 25,136,363
36 25,135,366
37 25,136,368
```

但是不难发现在采集到的数据中，容易出现，两组一模一样的数据，如果出现这种情况，会大大的影响到做回归曲线的，所以需要将相同的组删除掉。处理脚本如下所示：

```
1. import pandas as pd
```

Error! Unknown document property name.


```

2.
3.     # 定义数据
4.     data = pd.read_csv('test - 副本.csv')
5.
6.     # 创建 DataFrame
7.     df = pd.DataFrame(data)
8.
9.     # 去重
10.    unique_df = df.drop_duplicates(subset=['5_17', '0_12'])
11.
12.    # 根据所有列去除重复行
13.    unique_df_all = df.drop_duplicates()
14.
15.    # 导出数据到 CSV
16.    unique_df.to_csv('unique_data.csv', index=False)

```

导入模块:

pandas: 用于数据处理和分析。

定义数据:

data: 通过 `pd.read_csv('test - 副本.csv')` 读取 CSV 文件中的数据。这里假设 CSV 文件的名称是 `test - 副本.csv`。

创建 **DataFrame**:

df: 创建一个新的 **DataFrame** 对象, 用于存储从 CSV 文件中读取的数据。

去重操作:

unique_df: 使用 `drop_duplicates` 方法去除 df 中基于某些列的重复记录。

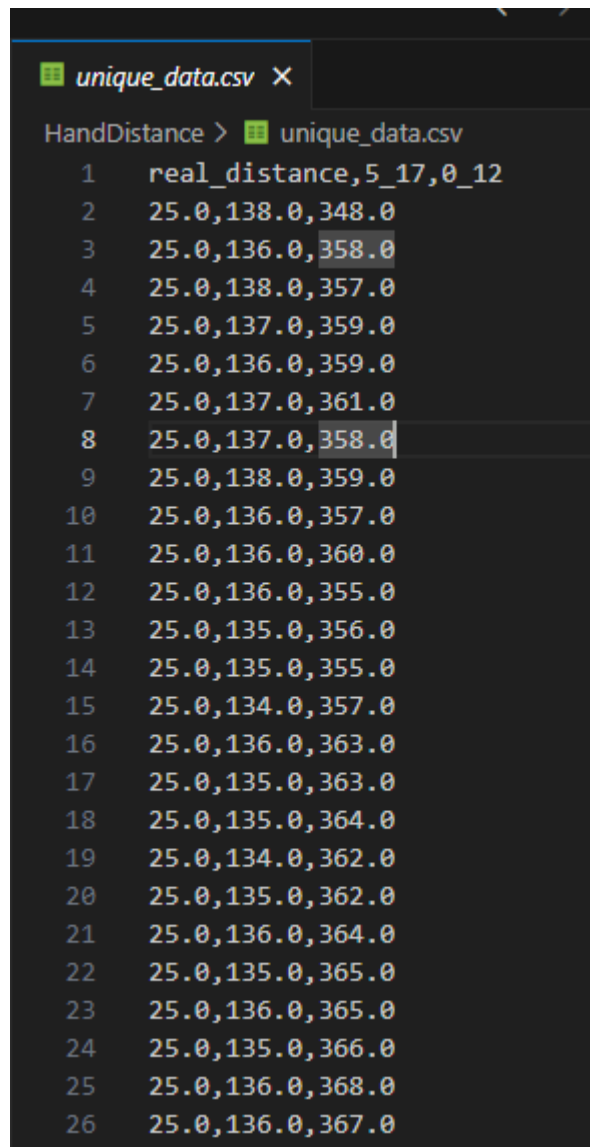
`subset=['5_17', '0_12']`参数指定了只考虑 5_17 和 0_12 这两列来识别重复行。如果这些列的组合在多行中完全相同，则只保留第一次出现的行。

根据所有列去除重复行：

`unique_df_all`：再次使用 `drop_duplicates` 方法，但这次没有指定 `subset` 参数，意味着将根据所有列来识别重复行。这样，只有当 **DataFrame** 中的所有列的值都完全相同时，才会被认为是重复的。

导出数据到 CSV：

`unique_df.to_csv('unique_data.csv', index=False)`：将去重后的 **DataFrame** `unique_df` 导出到名为 `unique_data.csv` 的新 CSV 文件中。`index=False` 参数表示在导出时不包括行索引。



	real_distance,5_17,0_12
1	25.0,138.0,348.0
2	25.0,136.0,358.0
3	25.0,138.0,357.0
4	25.0,137.0,359.0
5	25.0,136.0,359.0
6	25.0,137.0,361.0
7	25.0,137.0,358.0
8	25.0,138.0,359.0
9	25.0,136.0,357.0
10	25.0,136.0,360.0
11	25.0,136.0,355.0
12	25.0,135.0,356.0
13	25.0,135.0,355.0
14	25.0,134.0,357.0
15	25.0,136.0,363.0
16	25.0,135.0,363.0
17	25.0,135.0,364.0
18	25.0,134.0,362.0
19	25.0,135.0,362.0
20	25.0,136.0,364.0
21	25.0,135.0,365.0
22	25.0,136.0,365.0
23	25.0,135.0,366.0
24	25.0,136.0,368.0
25	25.0,136.0,367.0
26	

在得到了去重之后的数据，下一步将开始对数据进行回归处理，采用过的方式是 Ridge 回归模型，L2 正则的多项式

```
1. import numpy as np
2. import pandas as pd
3. from sklearn.linear_model import Ridge
4. from sklearn.model_selection import train_test_split, GridSearchCV
5. from sklearn.preprocessing import PolynomialFeatures
6. import matplotlib.pyplot as plt
7. from sklearn.pipeline import make_pipeline
```

Error! Unknown document property name.

```
8.
9. plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常
   显示中文标签
10. plt.rcParams['axes.unicode_minus'] = False # 用来正常
    显示负号
11.
12. data = pd.read_csv('unique_data.csv')
13. x1, x2 = data['5_17'], data['0_12']
14. y = data['real_distance']
15. X = np.column_stack((x1, x2))
16.
17. # 对 y 进行自然对数变换
18. y_log = np.log(y)
19.
20. # 创建多项式特征
21. poly = PolynomialFeatures(degree=2, include_bias=False)
22. X_poly = poly.fit_transform(X)
23.
24. # 分割数据
25. X_train, X_test, y_train_log, y_test_log = train_test_split(X_poly, y_log, test_size=0.2, random_state=42)
26.
27. # 使用 Ridge 回归模型
28. model = Ridge(alpha=1.0)
29. model.fit(X_train, y_train_log)
30.
31. # 预测
```

```

32. y_log_pred = model.predict(X_test)
33. y_pred = np.exp(y_log_pred)
34. r_square_log = model.score(X_test, y_test_log)
35. print("对数转换和多项式特征后的 Ridge 模型的
    R^2 Score:", r_square_log)
36.
37. # 输出模型的参数
38. coef = model.coef_
39. intercept = model.intercept_
40. print("模型的系数为:", coef)
41. print("模型的截距为:", intercept)
42.
43. # 创建一个管道, 包含多项式特征和Ridge 回归模型 L2 正则
44. pipeline = make_pipeline(PolynomialFeatures(degree=2, include_bias=False), Ridge(alpha=1.0))
45. pipeline.fit(X_train, y_train_log)
46.
47.
48. # 获取多项式特征的名称
49.
50. features = poly.get_feature_names_out(['x1', 'x2'])
51.
52. # 开始构建方程式
53. equation = "y = e^("
54. for i, feature in enumerate(features):
55.     if coef[i] > 0 and i > 0: # 处理正系数
56.         equation += " + "

```

```

57.         elif coef[i] < 0: # 处理负系数
58.             equation += " - "
59.             equation += f"{abs(coef[i]):.3f}*{feature}"
60.         equation += f" + {intercept:.3f})"
61.
62.     # 打印方程式
63.     print(equation)
64.
65.     # 绘图
66.     plt.scatter(X_test[:, 0], np.exp(y_test_log), color='blue', label='实际值')
67.     plt.scatter(X_test[:, 0], y_pred, color='red', alpha=0.5, label='预测值')
68.     plt.title('Ridge 回归的实际值与预测值\n' + equation)
69.     plt.xlabel('X1 (特征值)')
70.     plt.ylabel('Y (目标值)')
71.     plt.legend()
72.     plt.show()
73.
74.
75.     import json
76.
77.     # 构建模型数据和方程式的字典
78.     model_data = {
79.         "equation": equation,

```

```

80.         "description": "这是使用具有多项式特征的岭回归预测基于
           x1 和 x2 的实际距离的模型方程。",
81.         "r_square_log": r_square_log, # 存储 R^2 分数
82.         "model_parameters": {
83.             "coefficients": coef.tolist(), # 将 Numpy 数组转换
           为列表
84.             "intercept": intercept
85.         },
86.         "features": features.tolist() # 确保特征名称以适当的格
           式存储
87.     }
88.
89. # 写入模型数据到 JSON 文件
90. with open('ridge_regression_model.json', 'w') as file:
91.     json.dump(model_data, file, indent=4)
92.
93. print("模型方程式和参数已成功保存到 JSON 文件。")

```

数据预处理

数据读取：使用 pandas 库的 read_csv 函数从 CSV 文件中加载数据集。

特征选择：从数据集中选择两个特征列 5_17 和 0_12，以及目标列 real_distance。

数据变换：对目标变量 real_distance 进行自然对数变换，这是一种常用的数据预处理方法，有助于减少数据的偏度，使数据更接近正态分布，并且稳定方差。

特征工程

多项式特征：使用 `PolynomialFeatures` 从原始特征中生成二次多项式特征。

这可以增加模型的复杂度，使其能够捕捉特征之间的交互效应。

特征组合：使用 `numpy` 的 `column_stack` 将两个特征列组合成一个多维数组。

模型训练

数据分割：使用 `train_test_split` 将数据集随机分割为训练集和测试集，其中测试集占 20%，这种分割有助于评估模型在未见过的数据上的表现。

模型选择：选择岭回归模型（Ridge），这是一种带有 L2 正则化的线性回归模型，适用于特征数量多且相关的情况。

模型训练与评估

模型训练：在训练集上训练岭回归模型，`alpha` 参数设置为 1.0，这是正则化强度的参数，较小的 `alpha` 值意味着更强的正则化。

预测：使用训练好的模型在测试集上进行预测，并将预测结果从对数空间转换回原始空间。

评估指标：计算 R^2 分数，这是一个衡量模型拟合优度的指标，值越接近 1 表示模型预测效果越好。

模型解释

模型参数：输出模型的系数和截距，这些参数是模型方程式的关键组成部分。

方程式构建：根据多项式特征的名称和模型系数，构建模型的方程式，这有助于理解模型是如何工作的。

结果可视化

1. **散点图：**使用 `matplotlib` 绘制实际值和预测值的散点图，以直观展示模型的预测效果。

2. **图表信息:** 在图表中添加标题、标签和图例, 以提供更多的上下文信息。

模型保存

1. **模型数据字典:** 创建一个字典, 包含模型方程式、描述、 R^2 分数、模型参数和特征名称。
2. **JSON 文件:** 将模型数据字典保存为 JSON 文件, 这是一种轻量级的数据交换格式, 易于阅读和编写, 同时也易于与其他语言和工具进行交互。

岭回归 (Ridge Regression) 详解

1. **原理:** 岭回归通过在损失函数中添加一个正则化项 (L2 范数) 来减少模型复杂度, 避免过拟合。正则化项与模型系数的平方和成正比。
2. **数学表达:** 岭回归的优化目标是最小化以下损失函数:

$$Loss = ||X \times w - y||^2 + \alpha \times ||w||^2$$

其中, X 是特征矩阵, w 是模型系数向量, y 是目标值, α 是正则化参数。

多项式回归 (Polynomial Regression) 详解

- **原理:** 多项式回归是线性回归的一个扩展, 它通过增加特征的高次项或特征之间的交互项来拟合数据。
- **数学表达:** 如果我们有两个特征 x_1 和 x_2 , 二次多项式回归模型可以表示为:
$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_2^2 + w_5x_1x_2 + \dots$$
- 其中, w_i 是模型系数。

我们可以看到这段代码实现了一个典型的机器学习流程, 从数据预处理到模型训练、评估、解释和保存。代码中使用的岭回归和多项式特征是解决回归问题中的常用技术。

```
ridge_regression_model.json x
HandDistance > {} ridge_regression_model.json > ...
1
2 "equation": "y = e^(- 0.004*x1 - 0.007*x2 - 0.000*x1^2 + 0.000*x1 x2 - 0.000*x2^2 + 5.514)",
3 "description": "\u8fd9\u662f\u4f7f\u7528\u5177\u6709\u591a\u9879\u5f0f\u7279\u5f81\u7684\u5cad\u56de\u5f52\u9884\u6d4b\u57fa",
4 "r_square_log": 0.994877034295452,
5 "model_parameters": {
6   "coefficients": [
7     -0.004067305655731796,
8     -0.006693224559602325,
9     -0.00012876435160600512,
10    9.590659661135591e-05,
11    -1.1131910088514775e-05
12  ],
13   "intercept": 5.5139729579962085
14 },
15 "features": [
16   "x1",
17   "x2",
18   "x1^2",
19   "x1 x2",
20   "x2^2"
21 ]
22
```

上图是最终的到的方程式，保存在了 json 文件中

5.6. 建立测距方程式模型

得到了方程式之后，我们可以通过代码的方式，通过这个方程，来计算出距离了，实现方式如下：

```
1. import numpy as np
2. import json
3. import math
4.
5. def predict_distance(x1, x2, x3, x4, y1, y2, y3, y4) -> float:
6.     # with open('ridge_regression_model.json', 'r') as file:
7.     #     model_data = json.load(file)
8.
9.     model_data = {
10.         "equation": "y = e^(- 0.004*x1 - 0.007*x2 -
11.         0.000*x1^2 + 0.000*x1 x2 - 0.000*x2^2 + 5.514)",
12.         "description": "\u8fd9\u662f\u4f7f\u7528\u5177\u6709\u591a\u9879\u5f0f\u7279\u5f81\u7684\u5cad\u56de\u5f52\u9884\u6d4b\u57fa"
```

```

d4b\u57fa\u4e8e x1 \u54c x2 \u7684\u5b9e\u9645\u8ddd\u79bb\u7
684\u6a21\u578b\u65b9\u7a0b\u3002",
12.         "r_square_log": 0.994877034295452,
13.         "model_parameters": {
14.             "coefficients": [
15.                 -0.004067305655731796,
16.                 -0.006693224559602325,
17.                 -0.00012876435160600512,
18.                 9.590659661135591e-05,
19.                 -1.1131910088514775e-05
20.             ],
21.             "intercept": 5.5139729579962085
22.         },
23.         "features": [
24.             "x1",
25.             "x2",
26.             "x1^2",
27.             "x1 x2",
28.             "x2^2"
29.         ]
30.     }
31.
32.
33.     coefficients = np.array(model_data['model_parameters']['coefficients'])
34.     intercept = model_data['model_parameters']['intercept']
35.     # print(coefficients, intercept)
36.     """根据多项式特征计算预测距离"""

```

```

37.         distance1 = int(math.sqrt((y2 - y1) ** 2 + (x2 -
    x1) ** 2))
38.         distance2 = int(math.sqrt((y4 - y3) ** 2 + (x4 -
    x3) ** 2))
39.
40.         features = [distance1, distance2, distance1**2, dista
    nce1*distance2, distance2**2]
41.         features = np.array(features) # 确保特征是numpy 数组
42.         y_log_predicted = np.dot(features, coefficients) + in
    tercept
43.         distance_predicted = np.exp(y_log_predicted) # 取指数
    获得原始距离
44.         return distance_predicted

```

代码定义了一个名为 `predict_distance` 的函数，它接受八个参数，这些参数代表了手部关键点的坐标和计算出的距离。函数的目的是使用一个预先训练好的岭回归模型来预测实际距离。以下是对代码的详细分析：

函数定义：

`predict_distance(x1, x2, x3, x4, y1, y2, y3, y4)`: 函数接受八个参数，每两个参数一组，代表手部关键点的 `x` 和 `y` 坐标。

模型数据：

代码中使用了一个硬编码的 `model_data` 字典来模拟从 JSON 文件中加载模型数据的行为。在实际应用中，这部分代码会被注释掉，并使用 `with open...` 语句块来从文件中读取模型数据。

模型参数：

`coefficients`: 模型的系数, 是一个一维 `numpy` 数组。

`intercept`: 模型的截距。

`features`: 模型使用的特征名称列表。

计算手部关键点间距离:

使用 `math.sqrt` 和距离公式计算由关键点坐标确定的两段距离 `distance1` 和 `distance2`。

特征构造:

根据计算出的距离和模型的特征名称列表, 构造一个特征数组 `features`, 包括原始距离和距离的平方项。

预测逻辑:

使用构造的特征数组和模型系数, 通过点积运算 `np.dot(features, coefficients)` 计算对数变换后的距离。

加上截距 `intercept` 得到对数变换后的预测值 `y_log_predicted`。

使用指数函数 `np.exp` 将预测值从对数空间转换回原始空间, 得到预测的实际距离 `distance_predicted`。

返回值:

函数返回预测的实际距离 `distance_predicted`。

5.7. 建立角度方程式模型

角度的方程更加简单, 只需要得到像素, 与手部识别框的位置关系即可, 得到每个像素得到的角度值。代码如下所示:

```
1. def predict_angle(x_val, y_val):
2.     # 广角摄像头一半的角度
3.     camera_angle = 60 / 2
```

```

4.      # 图像中心的坐标
5.      center = [320, 240]
6.      # 手部矩形框与中心点的坐标差值, ( $\Delta x$ ,  $\Delta y$ )
7.      center_df = [center[0]-x_val, center[1]-y_val]
8.      # print(center_df)
9.      # 矩形框中心点坐标差 $\Delta x$  乘以每一个像素对应的广角摄像头的角度
      值
10.     angle_x = -center_df[0] * (camera_anlge / center[0])
11.     # angle_x = center_df[0] * (camera_anlge / center[0])
12.     # 同理可得垂直方向的
13.     angle_y = center_df[1] * (camera_anlge / center[1])
14.
15.     return [angle_x, angle_y]

```

`def predict_angle(x_val, y_val):`: 定义了一个函数 `predict_angle`，它接受两个参数 `x_val` 和 `y_val`，这些参数代表手部在图像中的 `x` 和 `y` 坐标。

`camera_anlge = 60 / 2`: 定义了广角摄像头水平方向一半的角度。这里假设摄像头的水平视场角是 60 度，因此一半的角度是 30 度。

`center = [320, 240]`: 定义了图像的中心点坐标。这里假设图像的分辨率是 640x480，因此中心点坐标是 (320, 240)。

`center_df = [center[0]-x_val, center[1]-y_val]`: 计算手部位置与图像中心点的坐标差值，即 Δx 和 Δy 。

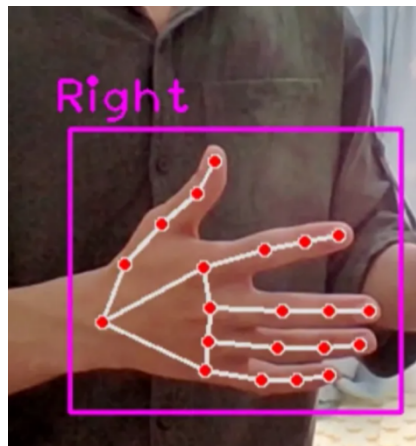
`angle_x = -center_df[0] * (camera_anlge / center[0])`: 根据手部与图像中心的水平距离 Δx 计算水平方向的角度偏差。这里使用了负号，表示水平方向的角度偏差与手部坐标的增加方向相反。

`# angle_x = center_df[0] * (camera_anlge / center[0]):`这一行被注释掉了,如果取消注释,将使用正号计算水平方向的角度偏差。

`angle_y = center_df[1] * (camera_anlge / center[1]):`根据手部与图像中心的垂直距离 Δy 计算垂直方向的角度偏差。

`return [angle_x, angle_y]:` 函数返回一个包含水平和垂直方向角度偏差的列表。

5.8. 模型本地测试



```
(base) root@davinci-mini:/opt/Dog/UDPControl# python HandDistance/main.py
Find 1 devices!

u3v device: [0]
device model name: MV-CS060-10UC-PRO
user serial number: DA0275184
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
距离: 76.08354297534035
角度 -3.375
距离: 67.79175314160902
角度 -1.78125
距离: 67.53255065873913
角度 -2.34375
距离: 69.22358033074413
角度 -1.6875
距离: 69.52555997559446
角度 -1.6875
距离: 69.52555997559446
角度 -1.59375
距离: 69.82730233095928
角度 -1.96875
```

根据测试信息可以得到距离与角度。(这里测试的值与图片所反馈的不符,在实际中我们需要自行测试,直到得到准确的值)

第六章 模型应用及其部署

6.1. 手部关键点检测

1.功能：手部关键点检测模型可以在输入图像中找到手的 21 个关键点，并返回这些点的坐标。

2.使用库：cvzone（封装了 MediaPipe 的功能）

3.检测流程：

a.初始化检测器：通过 HandDetector 类初始化手部检测器。

b.检测手部：使用 findHands 方法在每一帧图像中检测手部，并返回检测到的手部信息和处理后的图像。

c.提取关键点：从检测结果中提取 21 个关键点的坐标，用于进一步处理。

6.2. 部署流程

a.环境设置和依赖导入：设置 Python 环境并导入 OpenCV、cvzone 等依赖库。

b.初始化手部检测模型：使用 cvzone 封装的 MediaPipe 手部检测模型进行初始化。

c.图像捕获和处理：使用 OpenCV 实时捕获图像并传递给手部检测模型进行处理。

d.提取关键点和计算：从检测结果中提取手部关键点并进行相关计算。

e.结果显示：将检测结果和计算结果实时显示在图像上。

6.3. 导入必备的包

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  import cv2 # 图片处理三方库，用于对图片进行前后处理
```



```

5.  import logging
6.  from typing import *
7.  import sys
8.  sys.path.append('camera/')
9.  from HKcamera import getImage
10. from det_utils import get_labels_from_txt, letterbox, scale_coords, nms, draw_bbox # 模型前后处理相关函数
11.
12. # 导入定义的机器狗相关的包
13. from threading_utils.ThreadTemplates import * #
    线程的模板与基本轴指令的发送
14. from sendcommand import SendToCommand, heartbeat #
    发送简单指令与心跳包
15. from socketnetwork import network_utils #
    通信函数
16. from command.udp_command import * #
    存放各种结构体与状态数据、指令
17. from speeds.sportspeed import * #
    运动精准控制的基础版模型
18. from robotstatuswatcher.listener import * #
    监听机器狗状态

```

6.4. 建立通信

简单控制第一步：建立通信链接，socketnetwork/network_utils.py 文件定义了一个函数 `setup_socket_and_address`，用于创建一个 UDP 套接字，并设置机器狗的 IP 地址和端口号作为目标地址。该通信链接是所有后续控制和数据传输的

基础。不仅仅确保了指令传输的可靠性和及时性，还实现对机器狗的远程控制和监控以及为数据传输提供通道，代码如下：

```
1. # 导入 socket 模块，提供访问底层网络接口的功能
2. import socket
3.
4. # 从 typing 模块导入，用于类型提示和类型检查
5. from typing import *
6.
7. # 定义函数 setup_socket_and_address，用于创建套接字和设置目标地址
8. # 该函数接受目的地 IP 地址和端口作为参数，默认值分别为 '192.168.1.120' 和
   43893
9. # 函数返回值是一个包含套接字对象和目标地址元组的元组
10. def setup_socket_and_address(dest_ip='192.168.1.120', port=43893)
    -> Tuple[socket.socket, Tuple[str, int]]:
11.     # 创建 UDP 套接字，AF_INET 表示使用 IPv4 地址族，SOCK_DGRAM 表示使用
    UDP 协议
12.     sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
13.
14.     # 设置目标地址，由 IP 地址和端口号组成，封装成一个元组
15.     target_address = (dest_ip, port)
16.
17.     # 打印目标地址，这行代码被注释掉了
18.     # print(target_address)
19.
20.     # 返回创建的套接字和目标地址
21.     return sfd, target_address
```

这段Python代码定义了一个函数 `setup_socket_and_address`，其目的是设置一

个UDP套接字并返回该套接字及其目标地址。下面是对代码的逐行分析：

`import socket` - 导入Python的socket库，这个库提供了访问底层网络接口的机制。

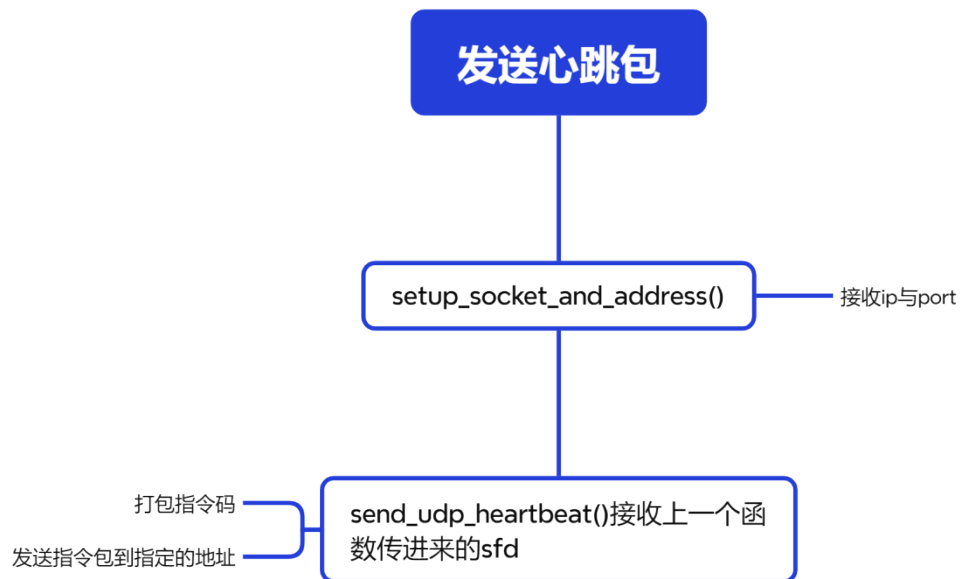
`from typing import *` - 从Python的typing模块导入所有类型提示，用于在函数和方法中添加类型注解，以提高代码的可读性和健壮性。

`def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) -> Tuple[socket.socket, Tuple[str, int]]:` - 定义了一个名为 `setup_socket_and_address` 的函数，它接受两个参数：`dest_ip` 和 `port`，这两个参数分别指定了目标IP地址和端口号，默认值分别是 `'192.168.1.120'` 和 `43893`。函数的返回类型被注解为 `Tuple[socket.socket, Tuple[str, int]]`，意味着它将返回一个元组，其中包含一个 `socket` 对象和一个包含IP地址和端口号的元组。

`sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` - 创建一个新的UDP套接字。`socket.AF_INET` 表示使用IPv4地址，`socket.SOCK_DGRAM` 表示使用UDP协议。

`target_address = (dest_ip, port)` - 将传入的IP地址和端口号组合成一个元组，这个元组将用作套接字的目标地址。

6.5. 发送心跳包



心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。这对于实时控制和数据传输至关重要。确保系统在运行过程中不会因为通信问题而出现故障，提高系统的整体可靠性。发送心跳包代码位于 `sendcommand/heartbeat.py`，代码如下：

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  import socket
5.  import struct
6.  import time
7.
8.  def send_udp_heartbeat(sfd, target_address, code=0x21040001, parameters_size=0, type=0, heartbeat_interval=0.25) -> None:
```

```

9.         # 打包心跳指令数据
10.         heartbeat_command = struct.pack('<III', code, parameters_size, type)
11.
12.         try:
13.             while True:
14.                 # 记录发送前的时间
15.                 start_time = time.time()
16.
17.                 # 发送心跳包
18.                 sfd.sendto(heartbeat_command, target_address)
19.
20.                 # 发送后的延时
21.                 time.sleep(max(0, heartbeat_interval -
22.                                (time.time() - start_time)))
22.
23.             except KeyboardInterrupt:
24.                 print("Heartbeat sending stopped by user.")
25.             finally:
26.                 # 关闭 socket
27.                 sfd.close()

```

`network_utils.setup_socket_and_address()` 函数被调用，用于创建一个UDP套接字（或其他类型的网络通信接口）并设置机器狗的IP地址和端口号作为目标地址。心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。导入所需的模块：`socket` 用于网络通信，`struct` 用于数据打包，`time` 用于时间相关的操作。

定义了 `send_udp_heartbeat` 函数，它接收以下参数：

`sfd`: 一个socket对象，用于发送数据。

`target_address`: 目标地址的元组，包含IP地址和端口号。

`code`: 心跳指令的代码，默认为 `0x21040001`。

`parameters_size`: 参数大小，默认为 `0`。

`type`: 心跳包的类型，默认为 `0`。

`heartbeat_interval`: 心跳包发送间隔，默认为 `0.25` 秒。

使用 `struct.pack` 将心跳指令的数据打包成字节串。参数 `<III` 表示使用小端格式打包三个整数。

在一个无限循环中，执行以下操作：

记录发送前的时间。

发送心跳包到目标地址。

计算发送后需要等待的时间，以确保心跳包的发送间隔符合 `heartbeat_interval`。

使用 `try-except` 结构来捕获 `KeyboardInterrupt` 异常，这通常发生在用户使用 `Ctrl+C` 中断程序时。如果捕获到这个异常，会打印一条消息表明心跳发送已被用户停止。

使用 `finally` 块来确保无论是否发生异常，`socket` 都会在函数结束时关闭。

启动心跳线程，代码如下：

```
1.    # 建立通讯
2.    sfd, target_address = network_utils.setup_socket_and_address()
3.
```

```

4.     # 定义发送心跳包的线程执行体
5.     # 注意*args 必须以 tuple 形式传入, 包括 sfd 和 target_address
6.     heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
7.     heartbeat_thread.start()

```

heartbeat_thread.start() 启动了心跳线程, 它将按照设定的时间间隔 (这里是0.25秒) 发送心跳包 (MyRepeatThread在后文中会解释)。

6.6. 速度函数

这段代码提供了一组用于控制设备运动的函数, 包括直线行驶、左右平移和左右旋转。

每个运动函数都接受一些参数, 如速度档位和运动的距离或时间, 并计算出所需的时间和速度。代码中使用了条件逻辑来根据不同的情况选择不同的计算公式。旋转函数中使用了栈跟踪来确定调用上下文, 代码是为了适应不同的使用场景而设计的。

```

1. #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  from command.udp_command import *                                # 存放各种结构
   体与状态数据、指令
5.
6.  def go_straight(long, speedgear=3, times = None):
7.      # 档位速度只是参考值, 具体速度按照实际来判断, 默认速度是3档
8.      speedgear_ditc = {
9.          1:7000,
10.         2:7500,
11.         3:8000,
12.         4:8700,
13.         5:9000,
14.         6:10500,
15.     }
16.
17.     val = speedgear_ditc[speedgear]

```

```

18.     # 如果传入的是时间, 计算出已经走过的距离, long 传入 9999 是为在特殊情况才
    能会有下面的情况
19.     if times != None and long == 9999:
20.         if long < 0:
21.             val = -val
22.             # speed_per_second_meter = (-1.086e-08) * val ** 2 -
                0.0003272 * val - 1.698
23.             speed_per_second_meter = (-1.5059e-08) * val ** 2 -
                (4.1944e-04) * val - 2.1804
24.             long = abs(times * speed_per_second_meter)
25.         else:
26.             speed_per_second_meter = (-7.237e-
                09) * val ** 2 + 0.0002933 * val - 1.643
27.             long = abs(times * speed_per_second_meter)
28.             return long
29.
30.     else:
31.         if long < 0:
32.             val = -val
33.             # speed_per_second_meter = (-1.086e-08) * val ** 2 -
                0.0003272 * val - 1.698
34.             speed_per_second_meter = (-1.5059e-08) * val ** 2 -
                (4.1944e-04) * val - 2.1804
35.             times = abs(long / speed_per_second_meter)
36.         else:
37.             speed_per_second_meter = (-7.237e-
                09) * val ** 2 + 0.0002933 * val - 1.643
38.             times = long / speed_per_second_meter
39.             return [times, val]
40.
41.
42. def translate_left_and_right(long, speedgear=3):
43.     # 档位速度只是参考值, 具体速度按照实际来判断, 默认速度是 3 档
44.     speedgear_ditc = {
45.         1: 14000,
46.         2: 18000,
47.         3: 21000,
48.         4: 24000,
49.         5: 27000,
50.         6: 34000,
51.     }
52.     val = speedgear_ditc[speedgear]
53.     if long < 0:
54.         val = -val

```



```

55.         speed_per_second_meter = -(1.6e-05) * val - 0.2256
56.         times = abs(long / speed_per_second_meter)
57.     else:
58.         speed_per_second_meter = (1.517e-05) * val - 0.1748
59.         times = long / speed_per_second_meter
60.     return [times, val]
61.
62.
63. def revolve_left_and_right(angle):
64.     def find_closest_output(input_value):
65.         # 判断角度是否超过了360度, 超过了则需要则算掉
66.         if input_value > 360:
67.             input_value = input_value / (input_value % 360)
68.
69.         # 输出到输入的映射字典
70.         output_to_input_map = {
71.             0: 0,
72.             15: 0.1,
73.             30: 0.2,
74.             45: 0.53,
75.             60: 0.6,
76.             75: 0.9,
77.             90: 1.3,
78.             105: 1.5,
79.             120: 1.9,
80.             135: 2.2,
81.             150: 2.3,
82.             165: 2.5,
83.             167: 2.8,
84.             185: 2.9,
85.             195: 3.1,
86.             210: 3.4,
87.             225: 3.7,
88.             240: 4,
89.             255: 4.3,
90.             270: 4.5,
91.             285: 4.7,
92.             300: 5,
93.             315: 5.3,
94.             330: 5.6,
95.             345: 5.8,
96.             360: 5.9,
97.         }
98.

```

```

99.
100.
101.     # 获取所有输出键并将其转换为列表
102.     keys = list(output_to_input_map.keys())
103.     # 寻找与输入值最接近的输出键
104.     closest_key = min(keys, key=lambda x: abs(x -
input_value))
105.     # 返回与最接近键关联的输入值
106.     return output_to_input_map[closest_key]
107.
108.     caller_function = StackTraceParser()
109.     # print(caller_function.get_formatted_stack_info())
110.
111.     # 根据栈的追踪, 如果旋转的调用执行不同的旋转方式
112.     if caller_function.get_stack_info()[0]['file'].split('/')[1] == 'hand_distance_main.py':
113.         if 8 < abs(angle) < 20:
114.             if angle < 0:
115.                 val = -10000
116.             else:
117.                 val = 10000
118.             times = 0.1
119.         elif 20 <= abs(angle):
120.             if angle < 0:
121.                 val = -10000
122.             else:
123.                 val = 10000
124.             times = 0.2
125.         else:
126.             times, val = 0, 0
127.
128.     else:
129.         times = find_closest_output(abs(angle)) # 获取时间与角度的
关系
130.         if angle < 0:
131.             val = -10000
132.         else:
133.             val = 10000
134.
135.     return [times, val]
136.
137.

```

```
go_straight(long, speedgear=3, times=None):
```

这个函数用于控制设备直线行驶。

`long`: 表示要行驶的距离，单位是米。

`speedgear`: 表示设备的档位，影响速度，其默认值为3。

`times`: 表示行驶时间，单位是秒。

函数首先定义了一个字典`speedgear_ditc`，它将档位映射到一个速度值，如果`times`不为None且`long`等于9999，函数会根据时间和速度计算行驶的距离。

如果`long`不是9999，函数会根据距离和速度计算所需的时间。

最终，函数返回行驶所需的时间（或距离）和速度值。

```
translate_left_and_right(long, speedgear=3):
```

这个函数用于控制设备左右平移。

参数和`go_straight`函数类似，但速度值是不同的字典`speedgear_ditc`。

根据`long`的正负，选择相应的速度计算公式来计算时间。

返回值是平移所需的时间和速度值。

```
revolve_left_and_right(angle):
```

这个函数用于控制设备左右旋转。

`angle`: 表示要旋转的角度，单位是度。

函数内部定义了一个辅助函数`find_closest_output`，用于找到最接近输入角度的输出值。

函数使用了一个映射字典`output_to_input_map`，将角度映射到一个输出值。

还有一个未定义的`StackTraceParser`类和它的`get_formatted_stack_info`方法，用于确定是哪个文件调用了旋转函数，以便执行不同的旋转逻辑。

根据调用的文件名和角度的大小，函数会设置不同的时间和速度值。

最终，函数返回旋转所需的时间和速度值。

6.7. 初始化手部检测器

实例化手部检测器，通过设置特定的置信度阈值和最大检测数量，创建了一个手部检测器实例，该实例能够用于后续的手部检测任务

- **detectionCon**: 这是一个参数，用于设置检测置信度的阈值。在这个例子中，阈值被设置为 0.8，意味着只有当检测算法的置信度达到或超过 80% 时，才会认为检测到的手部是有效的。这是一个质量控制措施，用于过滤掉那些置信度较低的检测结果，提高检测的准确性。
- **maxHands**: 这个参数设置了可以检测到手部的最大数量。在这个例子中，数量被设置为 2，表示该检测器会尝试在图像中找到最多两只手。

```
1.  # 实例化手部检测器
2.  detector = HandDetector(detectionCon=0.8, maxHands=2)
```

6.8. 建立两个标志位

```
1.  # 标志位
2.  HKflag = 1
3.  Dogflag = 0
```

Hkflag 的意义是进入到摄像头的循环中，表示还在测量距离，加入测量到了，那么回零，跳出摄像头，Dogflag 变为 1，进入到狗做动作的循环中去。

6.9. 摄像头测距

这段代码通过摄像头检测手部，计算其角度和距离，并根据距离阈值控制标志位的状态。

```
1.  # 无限循环，除非在循环内部被显式地中断
2.  while True:
```

```

3.      # 只要 HKflag 标志为真, 就继续循环
4.      while HKflag:
5.          # 获取图像, getImage() 函数需要自行定义或导入
6.          img = getImage()
7.
8.          # 使用 detector 的 findHands 方法检测图像中的手部
9.          # draw=False 表示不显示手部的检测框, 只关注实际距离的检测框
10.         hands, img = detector.findHands(img, draw=False)
11.
12.         # 初始化一个空列表, 用于存储某些数据
13.         list0 = []
14.
15.         # 如果检测到至少一只手
16.         if hands:
17.             # 获取第一只手的标记点列表
18.             lmList = hands[0]['lmList']
19.             # 获取第一只手的边界框坐标
20.             x, y, w, h = hands[0]['bbox']
21.
22.             # 从标记点列表中获取特定标记点的坐标
23.             # 这些标记点代表手部的关键位置
24.             x1, y1, z1 = lmList[5]
25.             x2, y2, z2 = lmList[17]
26.             x3, y3, z3 = lmList[0]
27.             x4, y4, z4 = lmList[12]
28.
29.             # 预测手部中心点的角度, predict_angle 函数需要自行定义或导
            入
30.             angle_xy = predict_angle(hands[0]['center'][0], hands
                [0]['center'][1])

```

Error! Unknown document property name.

```

31.
32.          # 根据标记点坐标预测距离, predict_distance 函数需要自行定
            义或导入
33.          distanceCM = predict_distance(x1, x2, x3, x4, y1, y2,
            y3, y4)
34.
35.          # 将预测的角度和距离存储到变量中
36.          angle = angle_xy[0] # 假设 predict_angle 返回的是一个
            包含角度的列表
37.
38.          # 如果距离超过10 厘米, 更新 HKflag 和 Dogflag 标志
39.          # 这里的条件被注释掉了, 因此实际上无论角度如何, 只要距离超
            过10 厘米就会更新标志
40.          if abs(distanceCM) > 10:
41.              HKflag = 0 # 更新 HKflag 标志
42.              Dogflag = 1 # 更新 Dogflag 标志
43.
44.          # 打印标志的状态
45.          # print('HKflag, Dogflag', HKflag, Dogflag)
46.
47.          # 打印角度和距离
48.          # print('角度和距离:', angle, distanceCM)
49.
50.          # 跳出循环, 结束手势检测
51.          break

```

while True: 一个无限循环, 将一直执行, 直到外部条件改变。

while HKflag: 一个内部循环, 条件是 **HKflag**。只要 **HKflag** 为真, 就会继续执行内部循环。

Error! Unknown document property name.

`img = getImage()`: 调用 `getImage` 函数获取图像。这个函数没有在代码中给出, 但我们可以假设它返回当前帧的图像。

`hands, img = detector.findHands(img, draw=False)`: 使用 `detector` 对象的 `findHands` 方法来检测图像中的手部。`draw=False` 参数意味着不显示手部的检测框, 只返回检测结果。

`list0 = []`: 初始化一个空列表 `list0`, 但在这段代码中没有进一步使用。

`if hands:`: 如果检测到至少一只手, 执行以下操作。

从检测到的第一只手中提取关键点坐标和边界框坐标。

`angle_xy = predict_angle(hands[0]['center'][0], hands[0]['center'][1])`: 调用 `predict_angle` 函数, 根据手部中心点的坐标计算角度偏差。

`distanceCM = predict_distance(x1, x2, x3, x4, y1, y2, y3, y4)`: 调用 `predict_distance` 函数, 根据手部关键点的坐标计算距离。

根据条件判断, 如果计算出的距离 `distanceCM` 的绝对值大于10厘米, 则改变 `HKflag` 和 `Dogflag` 的状态, 然后跳出内部循环。

`break`: 跳出内部循环。

6.10. 机器狗执行动作

这段代码控制机器狗执行动作, 通过循环和多线程处理动作命令。`while Dogflag`: 循环在 `Dogflag` 为真时执行, 进入循环后记录日志, 表明进入动作执行阶段。

```
1. # 只要 Dogflag 标志为真, 就持续循环
2. while Dogflag:
3.     # 记录进入循环的信息到日志中
4.     logging.info('-----进入狗循环中-----')
```

```

5.
6.     # 定义动作ID 与对应参数的映射字典
7.     actions_params = {
8.         1: [(distanceCM/100,)], # 动作ID 为1, 参数是距离 (以米为单位),
           使用默认速度
9.         2: [(angle,)] # 动作ID 为2, 参数是角度
10.    }
11.
12.     # 定义动作顺序列表, 包含动作ID 和参数列表索引
13.     # 动作将按照此顺序执行, 例如先执行动作ID 为2 的转弯动作, 再执行动作
       ID 为1 的前进动作
14.     actions_sequence = [(2, 0), (1, 0)]
15.
16.     # 遍历动作顺序列表, 执行每个动作
17.     for action_id, params_index in actions_sequence:
18.         # 根据动作ID 获取对应的动作函数, 如果不存在则为 None
19.         action_func = actions_dict.get(action_id, None)
20.         # 如果动作函数存在
21.         if action_func:
22.             # 从 actions_params 中获取指定动作ID 和参数索引对应的参数
23.             params = actions_params[action_id][params_index]
24.             # 记录动作函数和参数的详细信息到日志中
25.             logging.info('-----
           -----')
26.             logging.info(f'action_func: {action_func}')
27.             logging.info(f'params: {params}')
28.             # 检查参数是否为元组或列表
29.             if isinstance(params, tuple):
30.                 # 如果是, 调用动作函数, 传递参数, 并启动线程执行

```



```

31.         thread = action_func(*params) # 展开参数并传递给函数
32.         thread.start() # 启动线程
33.         thread.join() # 等待线程完成
34.         time.sleep(1) # 暂停1秒，用于等待动作完成或避免过快
           执行下一个动作
35.     else:
36.         # 如果参数不是元组或列表，记录错误日志
37.         logging.error('params 必须是一个 tuple 或者 列表')
38.
39.     # 更新 HKflag 和 Dogflag 标志的状态
40.     HKflag = 1 # 将 HKflag 设置为真
41.     Dogflag = 0 # 将 Dogflag 设置为假，从而退出循环

```

`while Dogflag:` 一个循环，只要 `Dogflag` 为真（非零或非空），就会持续执行循环体内的代码。

`logging.info('-----进入狗循环中-----')`: 使用 `logging` 库记录一条信息级别的日志，表明进入了循环。

`actions_params = { ... }`: 定义了一个字典 `actions_params`，它映射了动作ID到对应的参数列表。在这个例子中，有两个动作ID: 1 和 2，每个动作都有一个对应的参数。

`actions_sequence = [(2, 0), (1, 0)]`: 定义了一个列表 `actions_sequence`，包含了动作执行的顺序和参数索引。每个元素是一个元组，第一个值是动作ID，第二个值是 `actions_params` 中参数列表的索引。

`for action_id, params_index in actions_sequence:` 遍历 `actions_sequence` 列表，逐个处理动作。

`action_func = actions_dict.get(action_id, None)`: `actions_dict` 字典中根据当前的动作ID获取对应的动作函数。如果找不到, 则为 `None`。

`if action_func`: 如果成功获取了动作函数, 就继续执行。

`params = actions_params[action_id][params_index]`: 根据当前的动作ID和参数索引从 `actions_params` 获取参数。

`logging.info(...)`: 记录执行的动作函数和参数的日志信息。

`if isinstance(params, tuple)`: 检查参数是否为元组类型。

`thread = action_func(*params)`: 如果参数是元组, 将参数解包并传递给动作函数, 创建一个线程来执行该函数, 并启动线程。

`thread.start()`: 启动线程。

`thread.join()`: 等待线程执行完成。

`time.sleep(1)`: 等待一秒, 以确保动作执行的顺序和时间间隔。

`else`: 如果参数不是元组, 记录一条错误日志。

`HKflag = 1` 和 `Dogflag = 0`: 在循环的末尾, 修改了两个标志变量的值。这表示完成了某个任务, 或者切换了状态。

第七章 运行示例

首先在文件夹的目录下执行先执行这个指令

```
. /usr/local/Ascend/mxVision-5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh
```

然后再输入

```
python hand_distance_main.py
```

在对机器狗做出相对应的手势即可。

```
(base) root@davinci-mini:/opt/lab/HandTracking# python hand_distance_main.py
Find 1 devices!

u3v device: [0]
device model name: MV-CS060-10UC-PRO
user serial number: DA0275184
成功连接到ip: 192.168.1.100, port: 43897
Starting HeartbeatThread
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
INFO:root:-----进入狗循环中-----
INFO:root:-----
INFO:root:action_func: <function action_revolve_left_and_right at 0xe7ffec042040>
INFO:root:params: (-3.5625,)
Starting ACTION_revolve_left_and_right
INFO:root:ACTION_revolve_left_and_right由于超过时间阈值0秒，系统自动停止！
INFO:root:离开线程: ACTION_revolve_left_and_right
INFO:root:-----
INFO:root:action_func: <function action_go_straight at 0xe7ffec031e50>
INFO:root:params: (0.4200209919221258,)
Starting ACTION_pan_back_and_forth_thread
INFO:root:ACTION_pan_back_and_forth_thread由于超过时间阈值1.7483973489049192秒，系统自动停止！
INFO:root:离开线程: ACTION_pan_back_and_forth_thread
```